

---

# RTL Project - Final

## DDR2 DRAM Controller

---

25W URP Final Project



URP 1팀

# **CONTENTS**

---

**01 Bank Controller**

**02 Scheduler**

**03 Decoder**

**04 Testbench**

**05 Gate-Level Implementation**

---

# **Bank controller**

---

# Role of Bank controller

## 1 FSM 관리

: 현재 뱅크가 어떤 상태인지 추적 및 관리

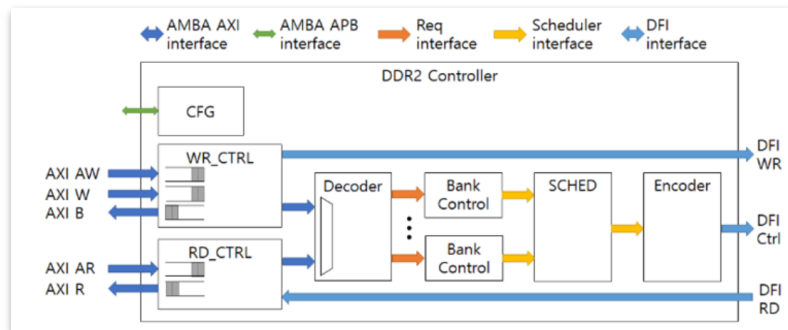
## 2 Timing constraints 준수

:  $t_{RP}$ ,  $t_{RCD}$  등 DRAM의 물리적인 한계로 인해 지켜야 하는 parameter를 만족하도록 관리

## 3

## Command Issue

: 다양한 조건들을 종합하여 DRAM에게 보낼 명령어를 생성  
bk\_reqs interface를 이용함





# Fundamental Principles of FSM Design

1 FSM은 **State Register, Next state logic, Output logic** 세 부분으로 나뉨

2

```
// Sequential logic for next state
always_ff @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        state      <= S_IDLE;
        cur_ra     <= '0;
        req_buf_valid <= 1'b0;
    end else begin
        state      <= state_n;
        cur_ra     <= cur_ra_n;
    end
end
```

2 현재 상태를 DFF에 저장  
: 반드시 clk과 rst\_n에 synchronous하고  
non-blocking assignment 이용

3

```
// Combinational logic
always_comb begin
    cur_ra_n      = cur_ra;
    state_n       = state;
    req_buf_valid_n = req_buf_valid;

    req_if_ready  = ~req_buf_valid;
end
```

3 현재 state와 input을 조합해 next state 결정  
: always\_comb 사용하고 blocking  
assignment 이용

# Sequential logic

```
// Sequential logic for next state
always_ff @(posedge clk or negedge rst_n) begin
    1 if(!rst_n) begin
        state          <= S_IDLE;
        cur_ra         <= '0;
        req_buf_valid  <= 1'b0;
    end else begin
    2     state          <= state_n;
        cur_ra         <= cur_ra_n;
        req_buf_valid  <= req_buf_valid_n;
        if(req_if.valid && req_if.ready) begin
            req_buf_wr    <= req_if.wr;
            req_buf_id    <= req_if.id;
            req_buf_len   <= req_if.len;
            req_buf_ra    <= req_if.ra;
            req_buf_ca    <= req_if.ca;
            req_buf_seq_num <= req_if.seq_num;
        end
    end
end
```

1 Reset 신호가 입력되면 초기화

2 기본적으로 현재 상태를 업데이트하고,  
Decoder와 Bank controller 모두  
준비되면 buffer 업데이트

# State

---

```
localparam S_IDLE = 3'b000, S_ACTIVATING = 3'b001, S_BANK_ACTIVE = 3'b010, S_READING = 3'b011,  
           S_WRITING = 3'b100, S_PRECHARGING = 3'b101, S_REFRESH = 3'b110;
```

# Combinational logic

```
//Combinational logic
always_comb begin
  1 cur_ra_n      = cur_ra;
    state_n      = state;
    req_buf_valid_n = req_buf_valid;

  2 req_if.ready   = ~req_buf_valid;
    ref_gnt_o     = 1'b0;

  3 bk_reqs = '0;
    bk_reqs.ba = BK_ID;

    if(req_if.valid && req_if.ready) begin
      req_buf_valid_n = 1'b1;
    end
end
```

1 Next state도 Current state 유지

2 이전에 받은 요청이 없거나 끝났으면  
준비 상태 유지

3 bk\_reqs를 초기화하고 Decoder와  
Bank controller 모두 준비되면  
valid flag 1로 설정

# Combinational logic

```
case (state)
  S_IDLE:begin
    1 if(ref_req_pending && is_t_rp_met && is_t_rc_met) begin
        bk_reqs.ref_req    = 1'b1;
        if(bk_gnts.ref_gnt) state_n = S_REFRESH;
      end
    2 else if (req_buf_valid && is_t_rc_met && is_t_rp_met) begin
        bk_reqs.act_req    = 1'b1;
        bk_reqs.ra         = req_buf_ra;
        bk_reqs.id         = req_buf_id;
        bk_reqs.len        = req_buf_len;
        bk_reqs.seq_num    = req_buf_seq_num;

        if(bk_gnts.act_gnt)begin
          state_n          = S_ACTIVATING;
          cur_ra_n          = req_buf_ra;
        end
      end
    end
  end
end

  S_ACTIVATING:begin
    3 if(is_t_rcd_met) begin
        state_n = S_BANK_ACTIVE;
      end
    end
  end
```

1 Refresh 요청 및 Timing 조건 만족하면 bk\_reqs의 req 설정

2 Write/Read controller로부터 요청이 들어오면 act req 설정 및 buffer에 정보 저장

3 Timing 조건만 만족하면 Bank active state로 이동

# Combinational logic

```
S_BANK_ACTIVE:begin
```

```
1 if(ref_req_pending && is_t_wtp_met && is_t_rtp_met && is_t_ras_met) begin  
  bk_reqs.pre_req = 1'b1;  
  if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
```

```
2 else if(req_buf_valid && (req_buf_ra != cur_ra))begin  
  if(is_t_ras_met && is_t_wtp_met && is_t_rtp_met)begin  
    bk_reqs.pre_req = 1'b1;  
    if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
```

```
end
```

```
3 else if(req_buf_valid && req_buf_wr == 1 && is_t_rcd_met && is_t_rtw_met)begin  
  bk_reqs.wr_req = 1'b1;  
  bk_reqs.ca = req_buf_ca;  
  bk_reqs.id = req_buf_id;  
  bk_reqs.len = req_buf_len;  
  bk_reqs.seq_num = req_buf_seq_num;  
  if(bk_gnts.wr_gnt)begin  
    state_n = S_WRITING;  
    req_buf_valid_n = 1'b0;
```

```
end
```

1 Bank active일 때 Refresh 요청이  
입력되면 Precharge

2 현재 open된 row와 일치하지 않으면  
Precharge 후 Activate

3 현재 open된 row와 일치할 때  
Timing 조건 만족하면 Write

# Combinational logic

---

```
end else if(req_buf_valid && req_buf_wr == 0 && is_t_rcd_met && is_t_wtr_met)begin
    bk_reqs.rd_req = 1'b1;
    bk_reqs.ca = req_buf_ca;
    bk_reqs.id = req_buf_id;
    bk_reqs.len = req_buf_len;
    bk_reqs.seq_num = req_buf_seq_num;
    if(bk_gnts.rd_gnt)begin
        state_n = S_READING;
        req_buf_valid_n = 1'b0;
    end
end
end
```

1 현재 open된 row와 일치할 때  
Timing 조건 만족하면 Read

# Combinational logic

1

```
S_READING:begin  
|   state_n = S_BANK_ACTIVE;  
end
```

```
S_WRITING:begin  
|   state_n = S_BANK_ACTIVE;  
end
```

2

```
S_PRECHARGING:begin  
|   if(is_t_rp_met)begin  
|       state_n = S_IDLE;  
|   end  
end
```

```
S_REFRESH:begin  
|   if(is_t_rfc_met)begin  
|       state_n = S_IDLE;  
|   end  
end
```

1

Read/Write는 곧바로 Bank active로 이동

2

Precharge와 Refresh는 timing 조건 만족하면 IDLE로 이동



# Combinational logic

---

```
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RC_WIDTH)) u_rc_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i(timing_if.t_rc_m1),.is_zero_o(is_t_rc_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RCD_WIDTH)) u_rcd_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i(timing_if.t_rcd_m2),.is_zero_o(is_t_rcd_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RP_WIDTH)) u_rp_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.pre_gnt),.reset_value_i(timing_if.t_rp_m2),.is_zero_o(is_t_rp_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RAS_WIDTH)) u_ras_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i(timing_if.t_ras_m1),.is_zero_o(is_t_ras_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RTP_WIDTH)) u_rtp_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.rd_gnt),.reset_value_i(timing_if.t_rtp_m1),.is_zero_o(is_t_rtp_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_WTP_WIDTH)) u_wtp_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.wr_gnt),.reset_value_i(timing_if.t_wtp_m1),.is_zero_o(is_t_wtp_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`ROW_OPEN_WIDTH)) u_row_open_cnt(.clk(clk),
.rst_n(rst_n),.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i(timing_if.row_open_cnt),.is_zero_o(is_t_row_open_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RFC_WIDTH)) u_rf_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.ref_gnt),.reset_value_i(timing_if.t_rfc_m2),.is_zero_o(is_t_rfc_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_WTR_WIDTH)) u_wtr_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.wr_gnt),.reset_value_i(timing_if.t_wtr_m1),.is_zero_o(is_t_wtr_met));
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RTW_WIDTH)) u_rtw_cnt(.clk(clk),.rst_n(rst_n),
.reset_cmd_i(bk_gnts.rd_gnt),.reset_value_i(timing_if.t_rtw_m1),.is_zero_o(is_t_rtw_met));
```

# Auto-Refresh

```
//Periodic Refresh
reg ref_req_pending;

always_ff @(posedge clk or negedge rst_n)begin
    if(!rst_n)begin
        ref_req_pending <= 1'b0;
    end else begin
        1 if(bk_reqs.ref_req && bk_gnts.ref_gnt)begin
            ref_req_pending <= 1'b0;
        end
        2 else if(ref_req_i)begin
            ref_req_pending <= 1'b1;
        end
    end
end
end
```

1 Refresh 이루어지면 flag 0으로 설정

2 DDR controller의 global clock으로부터  
신호를 받으면 flag 1로 설정

# Auto-Refresh

```
// Refresh Timer
localparam REF_INTERVAL_VAL = 32'd3120; // 7.8us / 2.5ns = 3120 cycles
logic [31:0] timer_ref;
logic ref_req;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        timer_ref <= REF_INTERVAL_VAL;
        ref_req <= 1'b0;
    end else begin
        if (sched_if.ref_gnt) begin
            timer_ref <= REF_INTERVAL_VAL;
            ref_req <= 1'b0;
        end else if (timer_ref == 0) begin
            ref_req <= 1'b1;
        end else begin
            timer_ref <= timer_ref - 1;
        end
    end
end
```

1 Refresh 완료하면 timer\_ref 초기화 및 ref\_req 0으로 설정

2 timer\_ref가 0이되면 Refresh 요청



일괄적인 Precharge를 위해 DDR Controller에 global clock 배치

# Auto-Refresh

```
genvar geni;

generate
  for (geni=0; geni<`DRAM_BK_CNT; geni=geni+1) begin : BK
    SAL_BK_CTRL
    #(
      .BK_ID          (geni)
    )
    u_bank_ctrl
    (
      .clk             (clk),
      .rst_n           (rst_n),

      .timing_if        (timing_if),

      .req_if           (bk_req_if_arr[geni]),

      .bk_reqs          (bk_if.reqs[geni]),
      .bk_gnts          (bk_if.gnts[geni]),

      .ref_req_i        (ref_req),
      .ref_gnt_o         ()
    )
  end
endgenerate
```

---

# Scheduler

---

# Requirement 1 : 타이밍 제약 준수

---

- 1 Requirement Define**  
: **bank~bank** 간의 **timing 제약** (tRRD, tCCD, tWTR, tRTW, tRFC) 준수를 위한 timing logic 제어
- 2 Timing Constraints Compliance**  
: 제공된 타이밍 카운터를 이용, **gnt** 신호를 기점으로 타이머 시작, 종료 후 **is\_zero** 신호를 통해 타이밍 통과 신호 생성
- 3 Arbitration Logic timing check**  
: 특정 req 신호가 들어왔을 때, 현재 **is\_zero** 신호가 만족 되었는지를 판단하여, **타이밍 제약이 충족 된 요청(req)인지 cand\_timing\_ok** 신호를 통하여 제어

# Requirement 1 : Code Implementation

```
1 // 2.1 tRRD
wire is_zero_trrd;
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RRD_WIDTH)) u_cnt_trrd (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.act_gnt), .reset_value_i
2 (timing_if.t_rrd_m1),
    .is_zero_o(is_zero_trrd));

// 2.2 tCCD
wire is_zero_tccd;
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_CCD_WIDTH)) u_cnt_tccd (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.rd_gnt | sched_if.
3 wr_gnt), .reset_value_i(timing_if.t_ccd_m1), .is_zero_o(is_zero_tccd));

// 2.3 tWTR
wire is_zero_twtr;
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_WTR_WIDTH)) u_cnt_twtr (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.wr_gnt), .reset_value_i
    (timing_if.t_wtr_m1),
    .is_zero_o(is_zero_twtr));

// 2.4 tRTW
wire is_zero_trtw;
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RTW_WIDTH)) u_cnt_trtw (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.rd_gnt), .reset_value_i
    (timing_if.t_rtw_m1),
    .is_zero_o(is_zero_trtw));

// 2.5 tRFC
wire is_zero_trfc;
SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RFC_WIDTH)) u_cnt_trfc (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.ref_gnt), .reset_value_i
    (timing_if.t_rfc_m1), .is_zero_o(is_zero_trfc));
```

1 **5가지 timing constraint counter**  
: tRRD, tCCD, tWTR, tRTW, tRFC 총 5개 카운터 생성

2 **Timer settings**  
: reset\_cmd\_i → 해당 타이밍의 gnt 신호로 타이머 동작 제어  
: reset\_value\_i → 실제 타이밍 제약 값을 입력

3 **Timer 완료 신호**  
: is\_zero 신호를 통해 타이밍 제약 통과 신호 생성

# Requirement 1 : Code Implementation

## <Arbitration logic timing check>

```
// B. Timing Check
cand_timing_ok = '0';
4 if (is_zero_trfc) begin
5   for(int k=0; k<bk_cnt; k++) begin
      if (req_acts[k])      cand_timing_ok[k] = is_zero_trrd;
      else if (req_rds[k]) cand_timing_ok[k] = is_zero_tccd &
is_zero_twtr;
      else if (req_wrs[k]) cand_timing_ok[k] = is_zero_tccd &
is_zero_trtw;
      else if (req_pres[k]) cand_timing_ok[k] = 1'b1;
    end
  end
```

```
6 always_comb begin
    // Base Set: Valid + Timing OK + ID Safe
    base_set = cand_valid & cand_timing_ok & id_ordering_safe;
```

4

### Refresh timing check

: 최상위 우선순위인 Refresh의 타이밍 체크를 통해 데이터 보호와 안정적인 Refresh 보장

5

5

### 요청(req)과 타이밍 제약

: req 신호와 is\_zero 신호의 비교를 통해 타이밍 제약 통과를 판단하고 이를 cand\_timing\_ok 신호를 통해 배출

6

6

### 최종 validity 판단

: 기타 제약 판단 기준과 함께 요청의 최종 validity 판단 신호인 base\_set으로의 연결



base\_set 신호는 최종 validity 신호로 rd/wr/pre/act req 신호, timing 제약 통과 신호, id ordering 통과 신호를 종합



## Requirement 2 : The Trade-off of Parallelism (병렬처리의 딜레마)

---

### 1 Requirement Define

: AXI 인터페이스의 **처리량(Throughput) 극대화**와 **순서(Ordering)의 보장** 두 가지 가치를 어떻게 동시에 잡을 것인가?

### 2 AXI 인터페이스의 ID와 Order 규칙

: AXI 프로토콜은 성능을 높이기 위해 **Out-of-Order Execution(비순차 실행)**을 허용하지만 **특정 요청에 대한 순차 처리 요구**를 위해 **동일 Transaction ID를 붙이고 요청을 보내는 순서를 조정**하여 이를 만족

핵심 규칙:

- **다른 ID끼리**: 순서가 뒤바껴도 상관없음 (**FIFO 원칙 적용**)
- **같은 ID끼리**: 반드시 요청한 **순서대로** 처리 (혹은 적어도 응답(Response)은 순서대로 와야 함) (**In-order 원칙 적용**)



요청의 실제 DRAM 주소에 상관 없이, 각 요청에 대하여 무조건 같은 ID가 있는지 비교 후 seq\_num 순서를 따짐

## Requirement 2 : Master의 요청 순서 준수 요구 Scenario

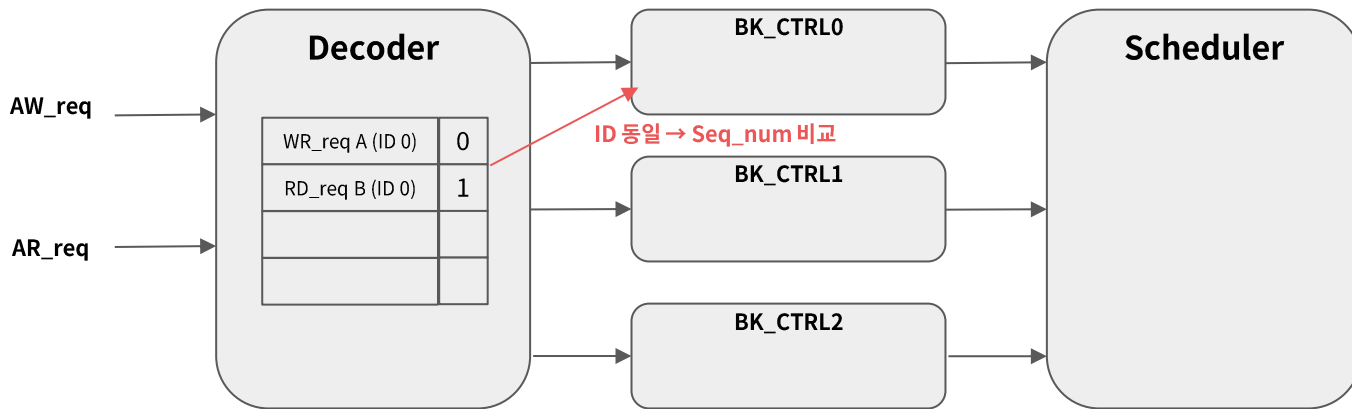
### 1 시나리오 1 : 서로 같은 DRAM addr 간 ID/seq\_num 비교 상황

상황 가정: Master가 데이터 무결성을 위해 특정 주소에 값을 쓴 후(Write), 바로 그 값을 읽어서(Read) 확인.  
(둘 다 ID=0 할당)

요청 순서:

1. Req A (Seq 0): WRITE (Bank 0)
2. Req B (Seq 1): READ (Bank 0 / 동일 addr)

→ Decoder의 ID/seq\_num 비교 logic을 통하여 예방 (후술 예정) (동일 Bank의 경우)



## Requirement 2 : Master의 요청 순서 준수 요구 Scenario

### 2 시나리오 2 : 서로 다른 DRAM addr 간 ID/seq\_num 비교 상황

#### ● Control Flag와 Data Payload (Dependency)

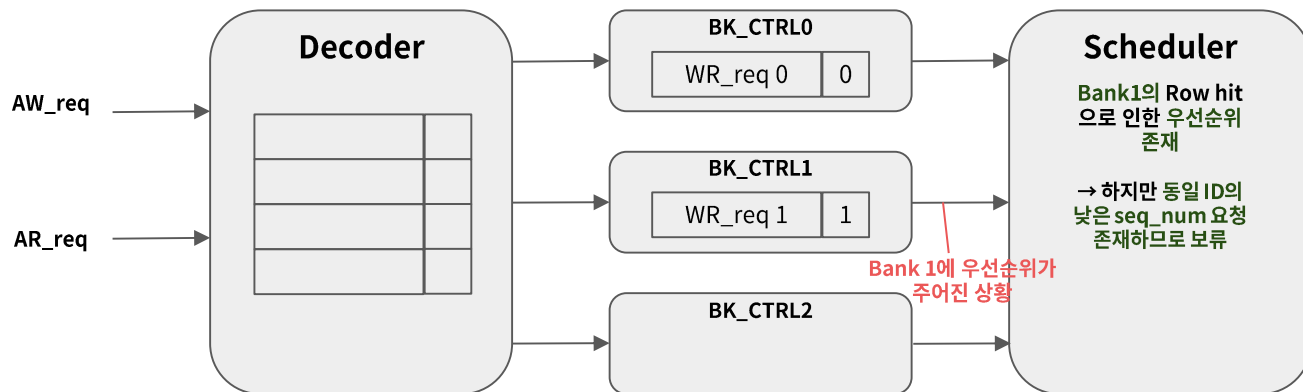
상황 가정: CPU가 어떤 데이터를 메모리에 쓰고(Data), 마지막에 "작업 끝"이라는 깃발(Flag)을 다른 주소에 세우려고 함.  
(둘 다 ID=1 할당)

요청 순서:

1. Req A (Seq 0): WRITE (대용량 데이터) → Bank 0
2. Req B (Seq 1): WRITE (완료 Flag 1비트) → Bank 1

(이때 만약 Bank 0이 바빠서 처리가 늦어지는데, Bank 1이 한가하다고 Flag를 먼저 써버리면 Signal Hazard 발생)

→ 스케줄러의 ID/seq\_num 비교 logic을 통하여 예방 (서로 다른 Bank의 경우)



# Requirement 2 : Code Implementation

1

```
// ID-Specific Ordering Check
logic [bk_cnt-1:0] id_ordering_safe;
```

```
// Check if any other bank has the SAME ID but a SMALLER seq_num (Older request)
always_comb begin
    id_ordering_safe = '1; // Default to safe (allow all)
```

2

```
for(int k=0; k<bk_cnt; k++) begin
    // For a candidate bank K
    if (bk_if.reqs[k].act_req || bk_if.reqs[k].rd_req || bk_if.reqs[k].wr_req || bk_if.reqs[k].pre_req) begin

        for(int j=0; j<bk_cnt; j++) begin
            if (k != j) begin
                // If bank J also has a request
                if (bk_if.reqs[j].act_req || bk_if.reqs[j].rd_req || bk_if.reqs[j].wr_req || bk_if.reqs[j].pre_req) begin
                    // And IDs match
```

3

```
if (bk_if.reqs[k].id == bk_if.reqs[j].id) begin
    // Wraparound-safe Comparison:
    // Check if J < K in a circular buffer sense.
    logic [7:0] diff;
    diff = bk_if.reqs[k].seq_num - bk_if.reqs[j].seq_num;
    if (diff < 8'd128 && diff != 0) begin
        id_ordering_safe[k] = 1'b0; // K must wait for J
    end
end
```

```
always_comb begin
    // Base Set: Valid + Timing OK + ID Safe
    base_set = cand_valid & cand_timing_ok & id_ordering_safe;
```

1

## Ordering safe 신호

: id\_ordering\_safe 신호를 통해 ID/seq\_num logic 통과 여부 반출

2

## 비교 대상 loop

: 이중 반복문(Nested Loop)을 돌며 "나(k)보다 먼저 처리되어야 할 요청(j)"이 있는가?"를 전수 조사

3

## seq\_num 비교 logic

Step 1: ID 매칭(`reqs[k].id == reqs[j].id`)

Step 2: Seq\_num 차이 계산 (`diff = k - j`)

Step 3: 유보 결정 (`diff < 128`)



단순 비교(`k > j`)가 아니라 뺄셈(-)을 하는 이유는 **Overflow(Wrap-around)** 때문 ("원형 큐" 개념으로 거리를 잴)

## Requirement 3-1 : Read/Write Batching

---

### 1 Requirement Define

: DRAM 데이터 버스의 **Bidirectional**함에 의한 물리적 제약 극복

**Read → Write**: 데이터 충돌을 막기 위해 꽤 긴 시간( $t_{RTW}$ )을 쉬어야 함.

**Write → Read**: 쓰기 완료 후 복구 시간( $t_{WTR}$ )을 기다려야 함.

▶ **Bus Turnaround Time (버스 방향 전환 시간)**

### 2 관성(Inertia)을 이용한 R/W Batching 구현

: 방금 실행한 명령이 무엇이었는지를 기억하여 "방금 했던 걸 계속 하려는 성질"을 logic으로 구현

### 3 우선순위 부여

: 이전 명령과 같은 종류의 요청에 우선권을 부여하여 **Tier 1 우선순위** 구현

# Requirement 3-1 : Code Implementation

1

```
// 3. Optimization Logic
// 3.1 Last Operation Direction

logic last_op_was_write;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        last_op_was_write <= 1'b0;
    end else begin
        if (sched_if.wr_gnt) begin
            last_op_was_write <= 1'b1;
        end else if (sched_if.rd_gnt) begin
            last_op_was_write <= 1'b0;
        end
        // Keep state on NOP or ACT/PRE
    end
end
```

2

```
if (last_op_was_write)
    cand_prio_dir = base_set & req_wrs;
else
    cand_prio_dir = base_set & req_rds;

// Final Selection Hierarchy
if (!cand_emergency) final_candidates = cand_emergency; //
Emergency_First!
else if (!cand_prio_dir) final_candidates = cand_prio_dir; // Same
Direction Second
else if (!cand_row_hit) final_candidates = cand_row_hit; // Then Any
Hit (Data Bus Open)
else
    final_candidates = base_set & ~req_rds &
    ~req_wrs; // Avoid RD/WR if not hit to prevent blind access
end
```

1

## last\_op\_was\_write

: 해당 신호를 통하여 wr\_gnt 신호가 1일 경우 true, rd\_gnt 신호가 1일 경우 false로 신호 반출

2

## batching용 후보 신호 생성

: cand\_prio\_dir 신호를 통하여 최종 validity 신호 (base\_set)와 wr/rd req가 모두 1(Pass) 인 경우 우선순위 후보군(Final Candidates)으로 인계



예외처리 (Starvation)는 후술 예정

## Requirement 3-2 : Row-Hit 우선

---

### 1 Requirement Define

: ACT/PRE 같은 준비 동작보다 **실제 데이터 전송 (RD/WR)**을 우선 처리해 Throughput을 높여야 함.

### 2 Candidate Grouping

: 기본 조건을 만족한 후보군 중에서 **RD/WR만 별도 그룹으로 분리**



row-hit일 경우 RD/WR이 준비 동작 없이 올라오므로, 그 자체를 유효한 데이터 전송 기회로 봄.

### 3 우선순위 부여

: Read/Write Batching 다음에 우선권을 부여하여 **Tier 2 우선순위** 구현

## Requirement 3-2 : Code Implementation

1 `cand_row_hit = base_set & (req_rds | req_wrs);`

```
// Final Selection Hierarchy
if (!cand_emergency)      final_candidates = cand_emergency;
// Emergency First!
else if (!cand_prio_dir)  final_candidates = cand_prio_dir;
// Same Direction Second
else if (!cand_row_hit)   final_candidates = cand_row_hit;
// Then Any Hit (Data Bus Open)
else                      final_candidates = base_set & ~req_rds & ~req_wrs;
// Avoid RD/WR if not hit to prevent blind access
```

1 데이터 우선 그룹 분리  
: base set 중 RD/WR만 선택

2 우선순위 배치  
: Emergency, Direction 유지 다음으로 RD/WR 선택



마지막 우선순위는 ACT/PRE에게 부여하여  
타이밍 제약으로 RD/WR이 안될 때 수행될  
수 있게 하였음.



## Requirement 3-3 : Starvation 방지

---

### 1 Requirement Define

: 특정 Bank의 요청이 계속 밀려 무한대기 하지 않도록 **최종적으로 보장**.

### 2 Wait time Tracking

: Bank별로 **요청이 있는데 grant 못 받은 시간** 카운트

### 3 Starved Detection & Emergency Candidate

: 타이머가 **Limit에 도달**하면 해당 Bank를 **기아상태로 선언**, 기아 상태 후보는 **최우선 후보군**으로 승격

### 4 우선순위 부여

: 가장 먼저 우선권을 부여하여 **Tier 0 우선순위** 구현

## Requirement 3-3 : Code Implementation

1

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for(int i=0; i<bk_cnt; i++) wait_timer[i] <= '0;
    end else begin
        for(int i=0; i<bk_cnt; i++) begin
            // If request exists...
            if (bk_if.reqs[i].rd_req || bk_if.reqs[i].wr_req || bk_if.reqs[i].act_req) begin
                // ...and NOT granted yet
                if (!bk_if.gnts[i].rd_gnt && !bk_if.gnts[i].wr_gnt && !bk_if.gnts[i].act_gnt) begin
                    // This prevents banks from being blocked indefinitely by a stream of R/W in
                    // other banks.
                    if (wait_timer[i] < STARVATION_LIMIT)
                        wait_timer[i] <= wait_timer[i] + 1;
                end else begin
                    wait_timer[i] <= '0; // Granted! Reset timer.
                end
            end else begin
                wait_timer[i] <= '0; // No request. Reset timer.
            end
        end
    end
end
```

1

### Bank별 대기시간 카운팅

: 요청이 있고 grant가 나가지 못할 경우 + 1  
: grant가 나가거나 요청이 사라지면 0으로 리셋

## Requirement 3-3 : Code Implementation

```
localparam STARVATION_LIMIT = 8'd16; // Fixed at 16 per user requirement
```

2

```
assign is_starved[0] = (wait_timer[0] == STARVATION_LIMIT);  
assign is_starved[1] = (wait_timer[1] == STARVATION_LIMIT);  
assign is_starved[2] = (wait_timer[2] == STARVATION_LIMIT);  
assign is_starved[3] = (wait_timer[3] == STARVATION_LIMIT);
```

```
cand_emergency = base_set & is_starved;
```

3

```
// Final Selection Hierarchy
```

```
if (!cand_emergency) final_candidates = cand_emergency;
```

```
// Emergency First!
```

```
else if (!cand_prio_dir) final_candidates = cand_prio_dir;
```

```
// Same Direction Second
```

```
else if (!cand_row_hit) final_candidates = cand_row_hit;
```

```
// Then Any Hit (Data Bus Open)
```

```
else final_candidates = base_set & ~req_rds & ~req_wrs;
```

```
// Avoid RD/WR if not hit to prevent blind access
```

2

### 기아 상태 검출

: STARVATION\_LIMIT 도달 시 is\_starved로 선언

: 기본 조건을 만족하고 기아상태라면 **emergency**로 선언

3

### 우선순위 배치

: 최우선 후보(cand\_emergency)로 선정하여 가장 먼저 선택

## Requirement 4 : Oldest First 최종 중재

---

### 1 Requirement Define

- : 여러 Bank 후보 중 **가장 오래 대기한 요청**을 우선 선택해 공정성을 확보해야함.
- : 특정 Bank가 계속 밀리는 Head-of-Line Blocking / Starvation을 줄이는 것이 목적.

### 2 Ordering Metric

- : 각 요청은 발행 시점에 **8-bit 전역 시퀀스 번호**를 부여받음.
- : 단순 값 비교가 아닌 원형 공간에서 더 이전에 발급된 값 = Older로 해석.

### 3 Winner Selection

- : 최종 후보군 중 가장 오래된 요청을 최종 승자로 결정

## Requirement 4 : Code Implementation

1

```
// Only if there are candidates
if (!final_candidates) begin
    logic [7:0] min_seq;
    int winner_idx;
    logic first_found;

    winner_idx = 0;
    first_found = 1'b0;
    min_seq = '0; // default (not actually used since first_found logic overwrites it)
```

2

```
for (int k=0; k<bk_cnt; k++) begin
    if (final_candidates[k]) begin
        if (!first_found) begin
            // First valid candidate we see becomes the temporary winner
            winner_idx = k;
            // Use a dummy variable or specific logic to get min_seq
            min_seq = bk_if.reqs[k].seq_num;
            first_found = 1'b1;
        end else begin
```

1 **최종 후보군 (final\_candidates)**  
: final\_candidates가 입력이 되어 **최종**  
**후보군끼리 비교**

2 **처음 들어온 후보 min\_seq 지정**  
: 처음 들어온 후보를 min\_seq로 지정후 first  
found를 1로 올리고 비교로직으로 넘어감.

## Requirement 4 : Code Implementation

3

```
logic [7:0] seq_k;  
logic [7:0] diff_k_min;  
  
seq_k = bk_if.reqs[k].seq_num;  
diff_k_min = seq_k - min_seq;  
  
if (diff_k_min >= 8'd128) begin  
    winner_idx = k;  
    min_seq = seq_k;  
end
```

4

```
winner_oh[winner_idx] = 1'b1;
```

3

**seq\_num 비교 로직**

: wrap around를 고려  
:  $\text{seq\_k} - \text{min\_seq} \geq 128 \rightarrow \text{winner\_idx}$  갱신

4

**최종 승자 선택**

: 최종적으로 한 Bank만 1로 세팅

## Requirement 5 : 리프레시 관리

---

### 1 Requirement Define

- : Refresh 동안 다른 명령은 발행되면 안됨.
- : **Rank 단위** Refresh 수행.

### 2 REF Grant Condition & Enforcement

- : बैंक별 ref\_req를 모아 전역 조건을 만듬.
- : 모든 बैं크가 준비되고  $t_{RFC}$ 를 만족하면 Refresh 수행.

### 3 $t_{RFC}$ Protection

- : REF를 발행하면 새로운 명령 발행을 중단.

## Requirement 5 : Code Implementation

1

```
assign all_ref_reqs[k]      = bk_if.reqs[k].ref_req;  
// Grant refresh to all banks when the scheduler decides to refresh
```

1

**Bank별 refresh 요청 집계**  
: all\_ref\_reqs 전역 조건 생성

2

```
assign sched_if.ref_gnt = (!is_zero_trfc) ? 1'b0 : (&all_ref_reqs);  
  
assign bk_if.gnts[k].ref_gnt = sched_if.ref_gnt;
```

2

**REF 발행 조건**  
: 모든 Bank REF 요청 +  $t_{RFC}$  만족

3

```
// B. Timing Check  
cand_timing_ok = '0;  
if (is_zero_trfc) begin  
    for(int k=0; k<bk_cnt; k++) begin  
        if (req_acts[k])      cand_timing_ok[k] = is_zero_trrd;  
        else if (req_rds[k])  cand_timing_ok[k] = is_zero_tccd &  
            is_zero_twtr;  
        else if (req_wrs[k])  cand_timing_ok[k] = is_zero_tccd &  
            is_zero_trtw;  
        else if (req_pres[k]) cand_timing_ok[k] = 1'b1;  
    end  
end
```

3

**Refresh시 다른명령 차단**  
: is\_zero\_trfc가 0이면 cand\_timing\_ok가 전부 0

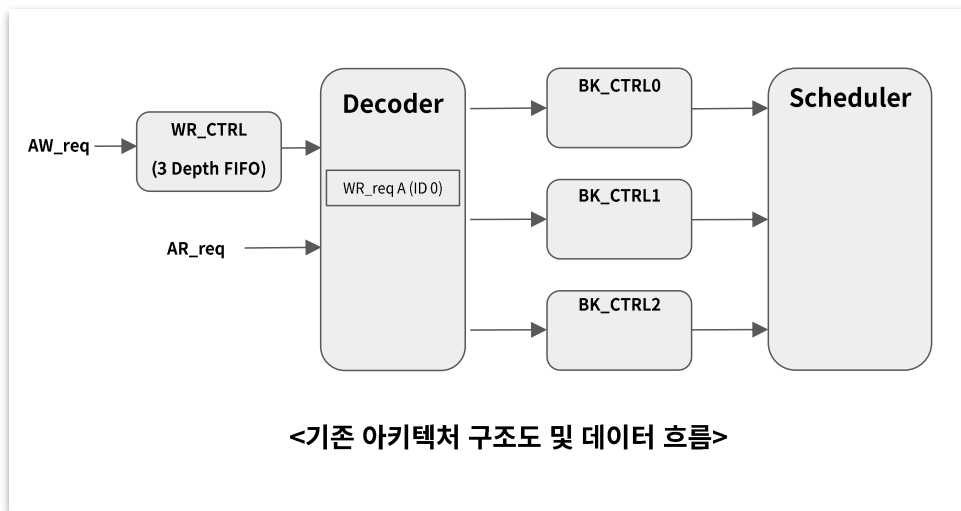


---

# Decoder

---

# Decoder : 기존 Decoder의 구조적 한계



## 1 Write 요청에 무조건적인 우선순위를 주는 구조

```
if (axi_aw_if.avalid) begin
    valid      = 1'b1;
    wr         = 1'b1;
    id         = axi_aw_if.aid;
end
else if (axi_ar_if.avalid) begin
    valid      = 1'b1;
    wr         = 1'b0;
    id         = axi_ar_if.aid;
end
```

: WR 요청에 의한 **RD 요청의 심각한 Starvation 문제** 발생 가능

## 2 Scheduler의 ID 기반 Requirement 충족 불가

: 먼저 들어오는 요청에 대한 global seq\_num 부여 기능 필요

## 3 AR 채널의 병렬성 저하

: 별도의 RD 요청 레지스터가 존재하지 않아, AR 채널의 병목 현상 발생 가능

# Decoder : 새롭게 설계한 Decoder의 특징

---

## 1 Reorder Buffer의 도입

: Read 요청을 받아줄 공간을 마련하여 부분적인 AR 채널의 병렬성 확보

## 2 엄격한 데이터 일관성 보장

1. 동일 ID에 대한 순서를 뒤바꿀 때 발생하는 데이터 오류를 방지하기 위해 요청 순서대로 고유의 Sequence Number를 부여
2. ROB에 요청을 채우는 방식으로 스케줄러의 '관성' 기법을 차용하여 RD starvation 문제 해결

## 3 뱅크 병렬성 극대화

: Reorder Buffer에 요청들을 저장해두고, 현재 타겟 뱅크가 Ready인 요청을 먼저 찾아 순서와 상관없이 발행  
→ 전체 시스템의 처리율 극대화와 공정성(Fairness) 확보



WR 컨트롤러의 경우, B\_resp 신호를 기존 방식(WR 컨트롤러의 FIFO에 들어오는 순간 반환)에서 디코더로 요청을 넘기는 순간 B\_resp 신호를 넘기는 방식으로 변경

# Decoder : 작동 원리

1

```
// Arbitration State
logic last_alloc_wr;
logic [3:0] starve_cnt;
localparam STARVE_LIMIT = 4'd8;
```

2

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        last_alloc_wr <= 1'b0;
        starve_cnt <= '0;
    end else begin
        // Update Direction & Starvation Counter
        if (aw_grant && axi_ar_if.avalid) begin
            // Granted Write while Read was waiting
            if (last_alloc_wr) begin
                if (starve_cnt < STARVE_LIMIT) starve_cnt <= starve_cnt + 1;
            end else begin
                starve_cnt <= 4'd1;
            end
            last_alloc_wr <= 1'b1;
        end else if (ar_grant && axi_aw_if.avalid) begin
            // Granted Read while Write was waiting
            if (!last_alloc_wr) begin
                if (starve_cnt < STARVE_LIMIT) starve_cnt <= starve_cnt + 1;
            end
        end
    end
end
```

3

```
if (!full) begin
    if (axi_ar_if.avalid && axi_aw_if.avalid) begin
        // Collision: Determine Priority
        if (starve_cnt >= STARVE_LIMIT) begin
            // Force Switch (Starvation Avoidance)
            if (last_alloc_wr) ar_grant = 1'b1; // Was Writing, force Read
            else
                aw_grant = 1'b1; // Was Reading, force Write
        end else begin
            // Inertia (Follow last direction)
            if (last_alloc_wr) aw_grant = 1'b1;
            else
                ar_grant = 1'b1;
        end
    end else if (axi_ar_if.avalid) begin
        ar_grant = 1'b1;
    end else if (axi_aw_if.avalid) begin
        aw_grant = 1'b1;
    end
end
```

1 AR과 AW 요청 간의 Batching logic용 Parameter 설정

2 starvation 카운터를 통한 공정성 확보  
: R/W간의 **ROB** 입장 횟수에 따라 카운터를 증가시켜 starvation logic 구현

3 last\_alloc\_wr 신호를 이용한 구현  
: 스케줄러와 마찬가지로 last\_alloc 신호를 통해 관성에 의한 R/W 우선순위 부여

# Decoder : 작동 원리

1

```
typedef struct packed {
    logic        valid;
    logic        wr;      // 1: Write, 0: Read
    axi_id_t      id;
    axi_len_t     len;
    axi_addr_t    addr;
    seq_num_t     seq_num;
    dram_ba_t     target_ba;
    dram_ra_t     target_ra;
    dram_ca_t     target_ca;
} rob_entry_t;

rob_entry_t rob[ROB_DEPTH];
```

2

```
// Ordering Check Logic (Hazard Detection)
logic [ROB_DEPTH-1:0] hazard_block;

always_comb begin
    hazard_block = '0;
    for(int k=0; k<ROB_DEPTH; k++) begin
        if (rob[k].valid) begin
            for(int j=0; j<ROB_DEPTH; j++) begin
                if (k != j && rob[j].valid) begin
                    if (rob[j].seq_num < rob[k].seq_num) begin
                        if (rob[j].id == rob[k].id)
                            hazard_block[k] = 1'b1;
                        if (rob[j].target_ba == rob[k].target_ba &&
                            rob[j].target_ra == rob[k].target_ra &&
                            rob[j].target_ca == rob[k].target_ca) begin
                            hazard_block[k] = 1'b1;
                        end
                    end
                end
            end
        end
    end
end
```

1

## Reorder Buffer 슬롯 구조

: 요청이 들어오면 바로 뱅크로 던지는 대신 내부의 슬롯 (ROB\_DEPTH) 중 비어있는 곳에 저장.

→ 뱅크 컨트롤러가 바쁘더라도 AXI 버스를 즉시 멈추지 않고, 계속해서 요청을 받아낼 수 있는 버퍼링 기능 수행

2

## seq\_num ID 비교 로직

: 현재 슬롯의 요청보다 seq\_num이 작은 유효 요청들 중, 동일한 주소를 접근하거나 동일한 ID 요청이 있는지 검사

→ 충돌이 감지되면 hazard\_block 플래그가 세워짐

# Decoder : 작동 원리

3

```
// Determine Candidates
always_comb begin
    for(int k=0; k<ROB_DEPTH; k++) begin
        can_issue[k] = 1'b0;
        if (rob[k].valid) begin
            // Condition 1: Target Bank is Ready
            if (bank_ready[rob[k].target_ba]) begin
                // Condition 2: No Hazard blocking
                if (!hazard_block[k]) begin
                    can_issue[k] = 1'b1;
                end
            end
        end
    end
end
```

3

## 발행 후보 선정

- ROB 슬롯이 유효함 (valid)
- 타겟 뱅크가 명령을 받을 준비가 됨 (bank\_ready)
- 데이터 해저드가 없음 (!hazard\_block)

4

```
rob_entry_t winner;
logic [$clog2(ROB_DEPTH)-1:0] winner_idx;
logic found_winner;

always_comb begin
    issue_grant_oh = '0;
    winner_idx = '0;
    found_winner = 1'b0;

    for(int k=0; k<ROB_DEPTH; k++) begin
        if (can_issue[k] && !rob[k].wr) begin
            issue_grant_oh[k] = 1'b1;
            found_winner = 1'b1;
            winner_idx = k[$clog2(ROB_DEPTH)-1:0];
            break;
        end
    end
end
```

```
if (!found_winner) begin
    for(int k=0; k<ROB_DEPTH; k++) begin
        if (can_issue[k]) begin
            issue_grant_oh[k] = 1'b1;
            found_winner = 1'b1;
            winner_idx = k[$clog2(ROB_DEPTH)-1:0];
            break;
        end
    end
end
```

4

## 최적 요청 승인

- 후보군 중 Read 요청을 최우선으로 선택하고, 없을 경우 Write 요청을 해당 Bank로 보냄

---

# Testbench

---

# Testbench

---



STEP 2 검증 환경에서 크게 3가지 주요 Testbench를 사용

1

## SAL\_TB\_TOP

: 메모리의 기본적인 읽기/쓰기 능력과 인터리빙 능력 검증  
→ 순차적 실행을 통한 인터리빙 & Row Conflict 상황 테스트

2

## SAL\_TB\_TOP\_REF

: Auto-Refresh 기능이 정상적으로 동작하는지 검증



# Testbench

---

## 3 SAL\_TB\_TOP\_rep2

: 무작위하고 복잡한 트래픽 상황에서 스케줄러의 중재 능력과 성능 검증

### 1) Test D (Random Row & Bank Conflict)

- Thread A : Bank 0에 대해 **Hit(Row 0) → Miss(Row 5) → Hit(Row 0)** 순서로 요청을 빠르게 발행
- Thread B : 동시에 Bank 1, Bank 2, Bank 3에 쓰기 요청을 발행

⇒ 단순 FIFO 구조라면 Hit → Miss → Hit 순서로 처리되지만, Hit → Hit → Miss 순서로 재정렬 하여  
전체 수행 시간이 단축 가능한지 확인

### 2) Test E (Heavy Random Starvation)

: 32개의 Read/Write 요청을 4개의 뱅크에 무작위로 쏟아 부음

# Testbench Result

---

 설계한 Decoder, Bank Controller, Scheduler의 성능을 평가하기 위해 크게 4가지 조합의 테스트를 진행

1 기존 Decoder & Round-Robin Scheduler(Baseline)

2 기존 Decoder & 설계한 Scheduler

3 설계한 Decoder & Round-Robin Scheduler(Baseline)

4 설계한 Decoder & 설계한 Scheduler

# Testbench Result : 결과 정리 도표

1 기존 Decoder & Round-Robin Scheduler(Baseline)

2 기존 Decoder & 설계한 Scheduler

3 설계한 Decoder & Round-Robin Scheduler(Baseline)

4 설계한 Decoder & 설계한 Scheduler

| <div> <div>TB</div> <div>Testing Sample</div> </div> | SAL_TB_TOP  | SAL_TB_TOP_rep2 |         |             | SAL_TB_TOP_REF |
|------------------------------------------------------|-------------|-----------------|---------|-------------|----------------|
|                                                      |             | Test D          | Test E  | Total       |                |
| 1                                                    | 1,338,750ps | 195ns           | 427.5ns | 1,771,250ps | O              |
| 2                                                    | 1,336,250ps | 197.5ns         | 427.5ns | 1,773,750ps | O              |
| 3                                                    | 1,358,750ps | 140ns           | 437.5ns | 1,723,750ps | O              |
| 4                                                    | 1,356,250ps | 140ns           | 435ns   | 1,721,250ps | O              |

## Testbench Result : 개선된 Decoder의 구조에 따른 타이밍 이득

|   | TB<br>Testing Sample | SAL_TB_TOP  | SAL_TB_TOP_rep2 |         |             |
|---|----------------------|-------------|-----------------|---------|-------------|
|   |                      |             | Test D          | Test E  | Total       |
| A | 1                    | 1,338,750ps | 195ns           | 427.5ns | 1,771,250ps |
|   | 2                    | 1,336,250ps | 197.5ns         | 427.5ns | 1,773,750ps |
| B | 3                    | 1,358,750ps | 140ns           | 437.5ns | 1,723,750ps |
|   | 4                    | 1,356,250ps | 140ns           | 435ns   | 1,721,250ps |

- 1 기존 Decoder & Round-Robin Scheduler (Baseline)
- 2 기존 Decoder & 설계한 Scheduler
- 3 설계한 Decoder & Round-Robin Scheduler (Baseline)
- 4 설계한 Decoder & 설계한 Scheduler



**Original TB와 Test E** 에서 개선된 Decoder가 더 느렸던 이유? (group A vs group B)  
: 개선된 Decoder의 경우 공정함을 위해 버스 방향 전환 비용을 지불하였기 때문.



**Test D**에서 개선된 Decoder가 더 빨랐던 이유? (group A vs group B)  
: 디코더에서의 병렬 처리를 통한 Bank 컨트롤러로의 요청 전달 방식이 정상 작동하였기 때문.

# Testbench Result Analysis : Scheduler의 구조에 따른 타이밍 이득

| Testing Sample \ TB | SAL_TB_TOP |
|---------------------|------------|
| 3                   | 1358750ps  |
| 4                   | 1356250ps  |

3 설계한 Decoder & Round-Robin Scheduler(Baseline)

4 설계한 Decoder & 설계한 Scheduler



Original TB에서 새롭게 설계한 스케줄러가 약 2ns 빨랐던 이유?

→ 빈 타이밍 상태에 **precharge 요청을 배치**하여 bubble을 줄였기 때문!

3

```
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
at time 691250.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000003 data = 99
at time 691250.0 ps INFO: Precharge bank 0
```

4

```
at time 683750.0 ps INFO: WRITE @ DQS= bank = 1 row = 0000 col = 00000003 data = 99
at time 683750.0 ps INFO: WRITE @ DQS= bank = 1 row = 0000 col = 00000003 data = 99
at time 683750.0 ps INFO: WRITE @ DQS= bank = 1 row = 0000 col = 00000003 data = 99
at time 683750.0 ps INFO: WRITE @ DQS= bank = 1 row = 0000 col = 00000003 data = 99
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 688750.0 ps INFO: Precharge bank 0
at time 690000.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000002 data = 99
at time 690000.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000002 data = 99
at time 690000.0 ps INFO: WRITE @ DQS= bank = 2 row = 0000 col = 00000002 data = 99
```

# Testbench Result Analysis : Scheduler의 구조에 따른 타이밍 이득

## 3 설계한 Decoder & Round-Robin Scheduler(Baseline)

## 4 설계한 Decoder & 설계한 Scheduler

| TB<br>Testing<br>Sample | SAL_TB_TOP_rep2 |         |             |
|-------------------------|-----------------|---------|-------------|
|                         | Test D          | Test E  | Total       |
| 3                       | 140ns           | 437.5ns | 1,723,750ps |
| 4                       | 140ns           | 435ns   | 1,721,250ps |



### Test D에서의 이점

- Round-Robin **Sample 3**: ID와 seq\_num의 검사 없이 실행 가능한 뱅크에 명령을 바로 입력함
- 설계한 Scheduler의 **Sample 4**: Row Hit을 통해 Precharge와 Activate 시간 절약 → ID 검사 시간 상쇄



### Test E에서의 이점

- Bus Turnaround 최소화: 무작위로 섞인 Read/Write 요청을 같은 종류끼리 모아서 처리하여 Turnaround 시간 제거

---

# Gate-Level Implementation

---

# Behavioral 코드와 Gate-level 코드의 차이점

## 1 RTL : 무엇을 할 것인가? (Behavioral)

- 1) 클럭 단위 동작만 정의, 물리적 지연은 고려 x
- 2) 특정 공정에 종속되지 않는 범용 코드
- 3) if-else, case, always\_ff 등의 High-Level 구문

```
always_comb begin
    cur_ra_n      = cur_ra;
    state_n       = state;
    req_buf_valid_n = req_buf_valid;

    req_if.ready   = ~req_buf_valid;
    ref_gnt_o      = 1'b0;

    bk_reqs = '0;
    bk_reqs.ba = BK_ID;

    if(req_if.valid && req_if.ready) begin
        req_buf_valid_n = 1'b1;
    end

    case (state)
        S_IDLE: begin
            if(ref_req_pending && is_trp_met && is_trc_met) begin
                bk_reqs.ref_req = 1'b1;
                if(bk_gnts.ref_gnt) state_n = S_REFRESH;
            end else if (req_buf_valid && is_trc_met && is_trp_met) begin
                bk_reqs.att_req = 1'b1;
                bk_reqs.ra      = req_buf_ra;
                // bk_reqs does not have id/len/seq_num based on this context?
            end
        end
    endcase

    //Periodic Refresh
    reg ref_req_pending;

    always_ff @ (posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            ref_req_pending <= 1'b0;
        end else begin
            if(bk_reqs.ref_req && bk_gnts.ref_gnt) begin
                ref_req_pending <= 1'b0;
            end
        end
    end
end
```

## 2 Gate-Level Netlist (Structural)

- 1) 논리 게이트가 구현됨 : INV / AND / NAND / OR / NOR / XOR / XNOR / MUX / DFF
- 2) 실제 지연 존재 (#을 통해 물리적 딜레이 구현)
- 3) assign 사용 불가, wire 와 generate 사용

```
module AND3 (A1,A2,A3,Y);
    output Y;
    input  A1,A2,A3;
    and #0.2 (Y,A3,A2,A1);
endmodule
```

```
module MUX21 (A1,A2,S0,Y);
    output Y;
    input  A1,A2,S0;
    assign #0.3 Y = S0 ? A1 : A2;
endmodule
```

```
wire n_req_buf_valid;
INV u_inv_valid (.A(req_buf_valid), .Y(n_req_buf_valid));
INV u_drive_rdy (.A(req_buf_valid), .Y(req_if.ready));
AND2 u_latch_en (.A1(req_if.valid), .A2(n_req_buf_valid), .Y(latch_enable));

// 4.2 Payload Registers (Using Generate + MUX + DFF)
genvar i;
generate
    for(i=0; i<AXI_ID_WIDTH; i=i+1) begin : gen_buf_id
        wire id_next;
        MUX21 u_mux_id (.A1(req_if.id[i]), .A2(req_buf_id[i]), .S0(latch_enable), .Y(id_next));
        DFF u_dff_id (.D(id_next), .RST_n(rst_n), .CLK(clk), .Q(req_buf_id[i]), .QN());
    end
end
```



# logic\_0 / logic\_1 생성 & assign 대체

1

```
// 0. Constant Generation (Logic 0 & Logic 1)
wire logic_0, logic_1;
AND2 u_tie_low (.A1(1'b0), .A2(1'b0), .Y(logic_0));
INV u_tie_high (.A(logic_0), .Y(logic_1));
```

2

```
wire B;
INV inv_1 (.A(A), .Y(B));
INV inv_2 (.A(B), .Y(C));

AND2 a (.A1(A), .A2(logic_1), .Y(C));
```

1

**logic\_0과 logic\_1 생성**  
- 0과 1을 상수화

2

**assign에 대한 해결책**  
- Buffer의 경우 wire를 하나 더 정의하고 식도 늘어나는 단점이 있다  
- **AND2를 사용하여** 한 줄로 간편하게

# FSM Encoding : One-Hot-Encoding

```
//our code
localparam S_IDLE = 3'b000, S_ACTIVATING = 3'b001, S_BANK_ACTIVE =
3'b010, S_READING = 3'b011, S_WRITING = 3'b100,
S_PRECHARGING = 3'b101, S_REFRESH = 3'b110;
```

```
// 1. Internal Wires Definition
wire [6:0] state;      // Current State (Q)
wire [6:0] state_n;    // Next State (D)
```

## 1 설계 복잡성의 최소화 (Implementation Efficiency)

- Normal Encoding의 경우 상태를 **2진수로 표현**
- **Gate-Level**의 복잡성이 증가하여 디버깅이 매우 어려움
- 논리 깊이를 획기적으로 줄여 Gate-Level Coding을 직관적이고 간결하게 변경

## 2 속도 및 타이밍 최적화 (High-Speed Performance)

- Normal Encoding은 상태 비트들이 압축되어 Combinational Logic을 복잡하게 만들 → 속도 저하의 원인
- One-Hot Encoding은 디코딩 로직이 거의 없어 Clock-to-Output 지연 시간이 훨씬 짧음

## 3 면적 대비 효율성 (The Cost Analysis)

- FSM의 State 개수는 7개
- Normal Encoding : **3개의 FF**
- One-Hot Encoding : **7개의 FF**
- Normal Encoding은 추가적인 **Decoder**가 더 필요함 → One-Hot Encoding이 면적 대비 효율성이 더 높음

# Key Implementation Challenges

---

## 1 Reset Value (FSM)

- RTL : `state <= S_IDLE;` (단순 할당)
- Gate : Standard Cell DFF는 **Reset시 0으로만 초기화**
- **FSM의 IDLE 상태는 1(One-Hot)**이어야 하는데 소자는 0으로 꺼짐

## 2 Parallelism (Combinational Logic)

- RTL : if-else 구문은 위에서 아래로 **순차적 평가**
- Gate : 전기는 모든 게이트에 **동시에 흐름**
- 동시다발적인 신호들 사이에서 **Arbitration**을 수행하는 회로를 설계해야 함

## 3 Persistence Conflict (Sequential Logic)

- RTL : **조건이 없으면 값을 유지**(latch implied)
- Gate : DFF는 클럭 엣지마다 무조건 입력을 샘플링함
- **데이터 유효성(Valid)이 없을 때 값을 덮어쓰지 않고 유지(Hold)**하게 만드는 Feedback Loop 필요

# Answer 1 : Inverted Storage Reset Logic

```
// Sequential logic for next state
always_ff @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        state      <= S_IDLE;
        cur_ra     <= '0;
        req_buf_valid <= 1'b0;
    end else begin
        state      <= state_n;
        cur_ra     <= cur_ra_n;
        req_buf_valid <= req_buf_valid_n;
        if(req_if.valid && req_if.ready) begin
            req_buf_wr      <= req_if.wr;
            req_buf_id      <= req_if.id;
            req_buf_len     <= req_if.len;
        end
    end
end
```



```
// 6. FSM State Registers

// 6.1 State[0] (IDLE): 리셋 시 1
wire d_idle_inverted;
wire q_idle_internal;

INV u_inv_idle_in (.A(state_n[0]), .Y(d_idle_inverted));
DFF u_dff_state_idle (
    .D(d_idle_inverted),
    .RST_n(rst_n),
    .CLK(clk),
    .Q(q_idle_internal),
    .QN()
);
INV u_inv_idle_out (.A(q_idle_internal), .Y(state[0]));
```

- 1 RTL : 단순 할당  
- state <= S\_IDLE;

- 2 Gate : Inverter Wrapping  
- DFF의 입력과 출력을 모두 반전 시켜 배치  
- Reset Active : DFF 내부 값은 0으로 초기화  
- Output Inverter : 내부 0이 INV를 통과하며 최종 출력 state[0]은 1이 됨

## Answer 2 : Priority Logic Arbitration

```
S_BANK_ACTIVE:begin
  if(ref_req_pending && is_t_wtp_met && is_t_rtp_met && is_t_ras_met) begin //
    bk_reqs.pre_req = 1'b1;
    if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
  end else if(req_buf_valid && (req_buf_ra != cur_ra))begin //
    if(is_t_ras_met && is_t_wtp_met && is_t_rtp_met)begin
      bk_reqs.pre_req = 1'b1;
      if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
    end
  end else if(req_buf_valid && req_buf_wr == 1 && is_t_rcd_met && is_t_rtw_met)begin
    write // Row Hit
    bk_reqs.wr_req = 1'b1;
    bk_reqs.ca = req_buf_ca;
    bk_reqs.id = req_buf_id;
    bk_reqs.len = req_buf_len;
    bk_reqs.seq_num = req_buf_seq_num; // Pass seq_num
    if(bk_gnts.wr_gnt)begin
      state_n = S_WRITING;
      req_buf_valid_n = 1'b0;
    end
  end
end
```



```
// 5.2 Priority Logic Signals

// 1. Refresh Priority Logic
wire cond_ref_needed, timer_ref_ready;
AND3 u_tm_ref (.A1(is_t_wtp_met), .A2(is_t_rtp_met), .A3(is_t_ras_met), .Y(timer_ref_ready));
AND2 u_cond_ref (.A1(ref_req_pending), .A2(timer_ref_ready), .Y(cond_ref_needed));

// 2. Row Miss Logic (Timing Aware)
wire n_cond_ref, miss_timers, raw_miss_req, cond_miss_needed;
INV u_inv_ref (.A(cond_ref_needed), .Y(n_cond_ref));
AND3 u_tm_miss (.A1(is_t_ras_met), .A2(is_t_wtp_met), .A3(is_t_rtp_met), .Y(miss_timers));
AND3 u_raw_miss (.A1(req_buf_valid), .A2(row_miss), .A3(miss_timers), .Y(raw_miss_req));
AND2 u_prio_miss (.A1(raw_miss_req), .A2(n_cond_ref), .Y(cond_miss_needed));
wire n_pure_row_miss;
INV u_inv_pure_miss (.A(row_miss), .Y(n_pure_row_miss));

// 3. Write Priority Logic
wire n_cond_miss_NOT_USED, wr_timers, raw_wr_req, cond_wr_needed;
INV u_inv_miss (.A(cond_miss_needed), .Y(n_cond_miss_NOT_USED));
AND2 u_tm_wr (.A1(is_t_rcd_met), .A2(is_t_rtw_met), .Y(wr_timers));
AND3 u_raw_wr (.A1(req_buf_valid), .A2(req_buf_wr), .A3(wr_timers), .Y(raw_wr_req));
AND3 u_prio_wr (.A1(raw_wr_req), .A2(n_cond_ref), .A3(n_pure_row_miss), .Y(cond_wr_needed));
```

### 1 RTL : if-else 구문

- 위에서부터 순차적으로 조건을 검사
- 상위 조건 만족시 하위 조건 자동 무시

### 2 Gate : Parallel Arbitration

- 상위 우선순위 신호 Refresh를 반전시켜 하위 로직 Miss의 AND 게이트 입력으로 연결
- Refresh가 1이면 n\_cond\_ref가 0이 되어 Row Miss 로직을 물리적으로 무력화(Masking)

# Answer 3 : Register Control (Enable Logic)

## 1 RTL

- Valid 신호가 있을 때만 데이터를 저장하고 없을 때는 기존 값을 Hold 해야 함
- else 구문 없이 작성하면 자동으로 유지됨

## 2 Gate : Explicit Feedback

### 1) Physical Constraint

- DFF는 Posedge마다 무조건 입력단 D의 값을 샘플링함
- 별도의 Enable 핀이 없는 DFF 사용시 데이터 유실 위험

### 2) MUX Recirculation

- 2:1 MUX를 DFF 앞단에 배치하여 **Feedback Loop** 형성
- **Enable = 1** : 새로운 데이터 (req\_if. ~) 선택
- **Enable = 0** : 기존 출력값 (req\_buf\_ ~)을 다시 입력으로 선택

```
if(req_if.valid && req_if.ready) begin
    req_buf_wr    <= req_if.wr;
    req_buf_id    <= req_if.id;
    req_buf_len   <= req_if.len;
    req_buf_ra    <= req_if.ra;
    req_buf_ca    <= req_if.ca;
    req_buf_seq_num <= req_if.seq_num; //Latch seq_num
end
```



```
// 4.1 Latch Enable & Ready Signal
wire n_req_buf_valid;
INV u_inv_valid (.A(req_buf_valid), .Y(n_req_buf_valid));
INV u_drive_rdy (.A(req_buf_valid), .Y(req_if.ready));
AND2 u_latch_en (.A1(req_if.valid), .A2(n_req_buf_valid), .Y(latch_enable));

// 4.2 Payload Registers (Using Generate + MUX + DFF)
genvar i;
generate
    wire wr_next;
    MUX21 u_mux_wr (.A1(req_if.wr), .A2(req_buf_wr), .S0(latch_enable), .Y(wr_next));
    DFF u_dff_wr (.D(wr_next), .RST_n(rst_n), .CLK(clk), .Q(req_buf_wr), .QN());

    for(i=0; i<AXI_ID_WIDTH; i=i+1) begin : gen_buf_id
        wire id_next;
        MUX21 u_mux_id (.A1(req_if.id[i]), .A2(req_buf_id[i]), .S0(latch_enable), .Y(id_next));
        DFF u_dff_id (.D(id_next), .RST_n(rst_n), .CLK(clk), .Q(req_buf_id[i]), .QN());
    end

    for(i=0; i<AXI_LEN_WIDTH; i=i+1) begin : gen_buf_len
        wire len_next;
        MUX21 u_mux_len (.A1(req_if.len[i]), .A2(req_buf_len[i]), .S0(latch_enable), .Y(len_next));
        DFF u_dff_len (.D(len_next), .RST_n(rst_n), .CLK(clk), .Q(req_buf_len[i]), .QN());
    end

    for(i=0; i<DRAM_RA_WIDTH; i=i+1) begin : gen_buf_ra
        wire ra_next;
        MUX21 u_mux_ra (.A1(req_if.ra[i]), .A2(req_buf_ra[i]), .S0(latch_enable), .Y(ra_next));
        DFF u_dff_ra (.D(ra_next), .RST_n(rst_n), .CLK(clk), .Q(req_buf_ra[i]), .QN());
    end
end
```

# Physical Consequence - Timing Delay

## 1 Behavioral Simulation : 1353750ps

```
Succeed!  
$finish called from file "/home/urp01/hyunsu/DDR2_Memory_Controller/script/../STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1353750.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1353750 ps
```



## 2 Gate-Level Simulation : 1361250ps

```
Succeed!  
$finish called from file "/home/urp01/chang/DDR2_Memory_Controller/script/../STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1361250.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1361250 ps
```

## 3 Why Delay Happens?

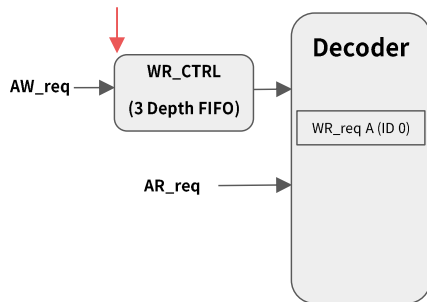
- 지연 시간을 줄이기 위해 **3-Gate 구현을 시도함**:  $t_{RFC}$  타이밍 규격을 만족하지 못해 violation 발생
- 근본 원인 : Behavioral Code 설계 시 **Safety Margin 없이 Ideal Timing에 맞춰 설계함**
- Gate-Level로 구현하는 과정에서 조금의 시간 증가가 확인됨

**THANK YOU**



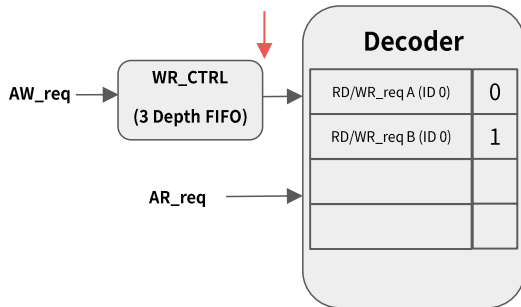
# APPENDIX : WR\_CTRL 개선 방안

WR\_CTRL FIFO 도달 순간 B\_resp 반환



<기존 데이터 흐름>

Decoder로 req 넘어가는 순간 B\_resp 반환



<개선 데이터 흐름>

- 1 AW의 B\_resp 신호 발생 시점의 문제점  
: WR 요청의 경우, WR\_CTRL의 FIFO를 거쳐 디코더로 도달하지만, RD 요청의 경우, 바로 Decoder로 연결되므로 **WR/RD 간의 요청 도달 시간차이 발생 가능**  
→ 이때 W 채널의 B\_resp 신호를 ID 부여 시점인 디코더에 맞추어 **WR 요청의 디코더 도달시 B\_resp 신호를 반환하여 명확한 요청 순서 보장**

- 2 AW의 B\_resp 신호 발생 시점 변경

```
// [수정 전]
.wren_i  (axi_w_if.wvalid & axi_w_if.wready & axi_w_if.wlast),
.wdata_i (axi_w_if.wid),

// [수정 후]
.wren_i  (axi_aw2_if.avalid & axi_aw2_if.aready), // 디코더가 AW를 수락했을 때
.wdata_i (axi_aw2_if.aid),                       // 전달된 요청의 ID를 사용
```

: 기존에는 Write controller가 data를 다 받았을 때 response 신호를 보냈지만, 수정 후에는 **axi\_aw2\_if를 통해 Decoder가 data를 받을 때 response 신호를 보냄**

# APPENDIX : Oldest-First Code

```
always_comb begin
    winner_oh = '0;

    // Only if there are candidates
    if (!final_candidates) begin
        logic [7:0] min_seq;
        int winner_idx;
        logic first_found;

        winner_idx = 0;
        first_found = 1'b0;
        min_seq = '0; // default (not actually used since first_found logic overwrites it)

        for (int k=0; k<bk_cnt; k++) begin
            if (final_candidates[k]) begin
                if (!first_found) begin
                    // First valid candidate we see becomes the temporary winner
                    winner_idx = k;
                    // Use a dummy variable or specific logic to get min_seq
                    min_seq = bk_if.reqs[k].seq_num;
                    first_found = 1'b1;
                end else begin
                    // We compare current k's seq (seq_k) vs min_seq
                    // We want to check: Is seq_k < min_seq (older)?
                    // Logic: wrapped diff (seq_k - min_seq)
                    // If result is small pos -> seq_k > min_seq (newer)
                    // If result is large pos (neg) -> seq_k < min_seq (older)

                    logic [7:0] seq_k;
                    logic [7:0] diff_k_min;

                    seq_k = bk_if.reqs[k].seq_num;
                    diff_k_min = seq_k - min_seq;

                    // If diff_k_min >= 128, it means seq_k is effectively smaller (older) due to wrap-around
                    // or just simple smaller number.
                    // Example: seq_k=255, min=0. diff = 255. 255 >= 128. True. k is older.
                    // Example: seq_k=0, min=255. diff = 1. False. k is newer.
                end
            end
        end
    end
end
```

```
        if (diff_k_min >= 8'd128) begin
            winner_idx = k;
            min_seq = seq_k;
        end
    end
end

winner_oh[winner_idx] = 1'b1;
end
```

# APPENDIX : Bank controller Code

```
1  `include "TIME_SCALE.svh"
2  `include "SAL_DDR_PARAMS.svh"
3
4  module SAL_BK_CTRL
5  #(
6      parameter          BK_ID    = 0
7  )
8  (
9      // clock & reset
10     input               clk,
11     input               rst_n,
12
13     // timing parameters
14     TIMING_IF.MON       timing_if,
15
16     // request from the address decoder
17     REQ_IF.DST         req_if,
18
19     // request to the scheduler
20     output bk_req_t     bk_reqs,
21     input  bk_gnt_t     bk_gnts,
22
23     // per-bank auto-refresh requests
24     input wire          ref_req_i,
25     output logic        ref_gnt_o
26 );
27
28 localparam S_IDLE = 3'b000, S_ACTIVATING = 3'b001, S_BANK_ACTIVE = 3'b010, S_READING = 3'b011,
29           S_WRITING = 3'b100, S_PRECHARGING = 3'b101, S_REFRESH = 3'b110;
30
31 logic [2:0] state, state_n;
32 logic req_buf_valid, req_buf_valid_n;
33 logic req_buf_wr;
34 logic [`AXI_ID_WIDTH-1:0] req_buf_id;
35 logic [`AXI_LEN_WIDTH-1:0] req_buf_len;
36 logic [`DRAM_RA_WIDTH-1:0] req_buf_ra, cur_ra, cur_ra_n;
```

```
37 logic [`DRAM_CA_WIDTH-1:0] req_buf_ca;
38 seq_num_t req_buf_seq_num; // [NEW] Added for ID Ordering
39
40 wire is_t_rc_met, is_t_rcd_met, is_t_rp_met, is_t_ras_met, is_t_rfc_met,
41      is_t_rtp_met, is_t_wtp_met, is_row_open_met, is_t_wtr_met, is_t_rtw_met;
42
43 //Periodic Refresh
44 reg ref_req_pending;
45
46 always_ff @(posedge clk or negedge rst_n)begin
47     if(!rst_n)begin
48         ref_req_pending <= 1'b0;
49     end else begin
50         if(bk_reqs.ref_req && bk_gnts.ref_gnt)begin
51             ref_req_pending <= 1'b0;
52         end
53         else if(ref_req_i)begin
54             ref_req_pending <= 1'b1;
55         end
56     end
57 end
58
59
60
61 // Sequential logic for next state
62 always_ff @(posedge clk or negedge rst_n) begin
63     if(!rst_n) begin
64         state <= S_IDLE;
65         cur_ra <= '0;
66         req_buf_valid <= 1'b0;
67     end else begin
68         state <= state_n;
69         cur_ra <= cur_ra_n;
70         req_buf_valid <= req_buf_valid_n;
71         if(req_if.valid && req_if.ready) begin
```

# APPENDIX : Bank controller Code

```
71         if(req_if.valid && req_if.ready) begin
72             req_buf_wr      <= req_if.wr;
73             req_buf_id      <= req_if.id;
74             req_buf_len     <= req_if.len;
75             req_buf_ra      <= req_if.ra;
76             req_buf_ca      <= req_if.ca;
77             req_buf_seq_num <= req_if.seq_num;
78         end
79     end
80 end
81
82
83
84
85 //Combinational logic
86 always_comb begin
87     cur_ra_n      = cur_ra;
88     state_n       = state;
89     req_buf_valid_n = req_buf_valid;
90
91     req_if.ready   = ~req_buf_valid;
92     ref_gnt_o      = 1'b0;
93
94     bk_reqs = '0;
95     bk_reqs.ba = BK_ID;
96
97     if(req_if.valid && req_if.ready) begin
98         req_buf_valid_n = 1'b1;
99     end
100
101     case (state)
102     S_IDLE:begin
```

```
103         if(ref_req_pending && is_t_rp_met && is_t_rc_met) begin
104             bk_reqs.ref_req = 1'b1;
105             if(bk_gnts.ref_gnt) state_n = S_REFRESH;
106         end else if (req_buf_valid && is_t_rc_met && is_t_rp_met) begin
107             bk_reqs.act_req = 1'b1;
108             bk_reqs.ra      = req_buf_ra;
109             bk_reqs.id      = req_buf_id;
110             bk_reqs.len     = req_buf_len;
111             bk_reqs.seq_num = req_buf_seq_num;
112
113             if(bk_gnts.act_gnt)begin
114                 state_n      = S_ACTIVATING;
115                 cur_ra_n     = req_buf_ra;
116             end
117         end
118     end
119
120     S_ACTIVATING:begin
121         if(is_t_rcd_met) begin
122             state_n = S_BANK_ACTIVE;
123         end
124     end
125
126     S_BANK_ACTIVE:begin
127         if(ref_req_pending && is_t_wtp_met && is_t_rtp_met && is_t_ras_met) begin
128             bk_reqs.pre_req = 1'b1;
129             if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
130         end else if(req_buf_valid && (req_buf_ra != cur_ra))begin
131             if(is_t_ras_met && is_t_wtp_met && is_t_rtp_met)begin
132                 bk_reqs.pre_req = 1'b1;
133                 if(bk_gnts.pre_gnt) state_n = S_PRECHARGING;
```

# APPENDIX : Bank controller Code

```

134         end
135     end else if (req_buf_valid && req_buf_wr == 1 && is_t_rcd_met && is_t_rtw_met) begin
136         bk_reqs.wr_req = 1'b1;
137         bk_reqs.ca = req_buf_ca;
138         bk_reqs.id = req_buf_id;
139         bk_reqs.len = req_buf_len;
140         bk_reqs.seq_num = req_buf_seq_num;
141         if (bk_gnts.wr_gnt) begin
142             state_n = S_WRITING;
143             req_buf_valid_n = 1'b0;
144         end
145     end else if (req_buf_valid && req_buf_wr == 0 && is_t_rcd_met && is_t_wtr_met) begin
146         bk_reqs.rd_req = 1'b1;
147         bk_reqs.ca = req_buf_ca;
148         bk_reqs.id = req_buf_id;
149         bk_reqs.len = req_buf_len;
150         bk_reqs.seq_num = req_buf_seq_num;
151         if (bk_gnts.rd_gnt) begin
152             state_n = S_READING;
153             req_buf_valid_n = 1'b0;
154         end
155     end
156 endS
157
158 S_READING: begin
159     state_n = S_BANK_ACTIVE;
160 end
161
162 S_WRITING: begin
163     state_n = S_BANK_ACTIVE;

```

```

164     end
165
166     S_PRECHARGING: begin
167         if (is_t_rp_met) begin
168             state_n = S_IDLE;
169         end
170     end
171
172     S_REFRESH: begin
173         if (is_t_rfc_met) begin
174             state_n = S_IDLE;
175         end
176     end
177 endcase
178
179 end
180
181 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RC_WIDTH")) u_rc_cnt(.clk(clk), .rst_n(rst_n),
182     .reset_cmd_i(bk_gnts.act_gnt), .reset_value_i(timing_if.t_rc_m1), .is_zero_o(is_t_rc_met));
183 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RCD_WIDTH")) u_rcd_cnt(.clk(clk), .rst_n(rst_n),
184     .reset_cmd_i(bk_gnts.act_gnt), .reset_value_i(timing_if.t_rcd_m2), .is_zero_o(is_t_rcd_met));
185 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RP_WIDTH")) u_rp_cnt(.clk(clk), .rst_n(rst_n),
186     .reset_cmd_i(bk_gnts.pre_gnt), .reset_value_i(timing_if.t_rp_m2), .is_zero_o(is_t_rp_met));
187 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RAS_WIDTH")) u_ras_cnt(.clk(clk), .rst_n(rst_n),
188     .reset_cmd_i(bk_gnts.act_gnt), .reset_value_i(timing_if.t_ras_m1), .is_zero_o(is_t_ras_met));
189 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RTP_WIDTH")) u_rtp_cnt(.clk(clk), .rst_n(rst_n),
190     .reset_cmd_i(bk_gnts.rd_gnt), .reset_value_i(timing_if.t_rtp_m1), .is_zero_o(is_t_rtp_met));
191 SAL_TIMING_CNTR #(CNTR_WIDTH("T_WTP_WIDTH")) u_wtp_cnt(.clk(clk), .rst_n(rst_n),
192     .reset_cmd_i(bk_gnts.wr_gnt), .reset_value_i(timing_if.t_wtp_m1), .is_zero_o(is_t_wtp_met));
193 SAL_TIMING_CNTR #(CNTR_WIDTH("ROW_OPEN_WIDTH")) u_row_open_cnt(.clk(clk),
194     .rst_n(rst_n), .reset_cmd_i(bk_gnts.act_gnt), .reset_value_i(timing_if.row_open_cnt), .is_zero_o(is_t_row_open));
195 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RFC_WIDTH")) u_rfc_cnt(.clk(clk), .rst_n(rst_n),
196     .reset_cmd_i(bk_gnts.ref_gnt), .reset_value_i(timing_if.t_rfc_m2), .is_zero_o(is_t_rfc_met));
197 SAL_TIMING_CNTR #(CNTR_WIDTH("T_WTR_WIDTH")) u_wtr_cnt(.clk(clk), .rst_n(rst_n),
198     .reset_cmd_i(bk_gnts.wr_gnt), .reset_value_i(timing_if.t_wtr_m1), .is_zero_o(is_t_wtr_met));
199 SAL_TIMING_CNTR #(CNTR_WIDTH("T_RTW_WIDTH")) u_rtw_cnt(.clk(clk), .rst_n(rst_n),
200     .reset_cmd_i(bk_gnts.rd_gnt), .reset_value_i(timing_if.t_rtw_m1), .is_zero_o(is_t_rtw_met));
201
202 endmodule // SAL_BK_CTRL_B

```

# APPENDIX : ADDR Decoder Code

```
1  `include "TIME_SCALE.svh"
2  `include "SAL_DDR_PARAMS.svh"
3
4  // Out-of-Order Dispatch Decoder with Reorder Buffer
5  // Solves Head-of-Line Blocking issue of simple schedulers.
6
7  module SAL_ADDR_DECODER
8  ✎ #(
9      parameter integer ROB_DEPTH = 4 // Number of slots (buffers)
10  )
11  ✎ (
12      // clock & reset
13      input          clk,
14      input          rst_n,
15
16      // request from the AXI side
17      AXI_A_IF.DST   axi_ar_if,
18      AXI_A_IF.DST   axi_aw_if,
19
20      // requests to bank controllers
21      REQ_IF.SRC     req_if_arr[`DRAM_BK_CNT]
22  ✎ );
23
24      // 1. Definition of Slot Entry
25  ✎ typedef struct packed {
26      logic          valid;
27      logic          wr;          // 1: Write, 0: Read
28      axi_id_t       id;
29      axi_len_t      len;
30      axi_addr_t     addr;
31      seq_num_t      seq_num;
32      dram_ba_t      target_ba;
33      dram_ra_t      target_ra;
34      dram_ca_t      target_ca;
35  } rob_entry_t;
36
```

```
37  rob_entry_t rob[ROB_DEPTH];
38
39      // Sequence Number Management
40  seq_num_t global_seq_num;
41
42      // 2. Input Logic (Allocation)
43
44      // Find first empty slot logic
45  logic [$clog2(ROB_DEPTH)-1:0] alloc_ptr;
46  logic          alloc_valid;
47
48      // Check if we have space
49  logic full;
50  logic [ROB_DEPTH-1:0] valid_vec;
51
52  genvar i;
53  ✎ generate
54      for(i=0; i<ROB_DEPTH; i++) assign valid_vec[i] = rob[i].valid;
55  endgenerate
56  assign full = &valid_vec;
57
58      // AXI Ready signals Generation (Arbitration: Inertia + Starvation
59      // Prevention)
60  logic aw_grant, ar_grant;
61
62      // Arbitration State
63  logic          last_alloc_wr;
64  logic [3:0] starve_cnt;
65  localparam STARVE_LIMIT = 4'd8;
66
67  ✎ always_ff @(posedge clk or negedge rst_n) begin
68      ✎ if (!rst_n) begin
69          last_alloc_wr <= 1'b0;
70          starve_cnt <= '0;
71      end else begin
```

# APPENDIX : ADDR Decoder Code

```
71 // Update Direction & Starvation Counter
72 if (aw_grant && axi_ar_if.avalid) begin
73     // Granted Write while Read was waiting
74     if (last_alloc_wr) begin
75         if (starve_cnt < STARVE_LIMIT) starve_cnt <=
            starve_cnt + 1;
76     end else begin
77         starve_cnt <= 4'd1;
78     end
79     last_alloc_wr <= 1'b1;
80 end else if (ar_grant && axi_aw_if.avalid) begin
81     // Granted Read while Write was waiting
82     if (!last_alloc_wr) begin
83         if (starve_cnt < STARVE_LIMIT) starve_cnt <=
            starve_cnt + 1;
84     end else begin
85         starve_cnt <= 4'd1;
86     end
87     last_alloc_wr <= 1'b0;
88 end else if (aw_grant) begin
89     // Granted Write (No Read waiting)
90     last_alloc_wr <= 1'b1;
91     starve_cnt <= '0;
92 end else if (ar_grant) begin
93     // Granted Read (No Write waiting)
94     last_alloc_wr <= 1'b0;
95     starve_cnt <= '0;
96 end
97 end
98
99
```

```
100 always_comb begin
101     aw_grant = 1'b0;
102     ar_grant = 1'b0;
103
104     if (!full) begin
105         if (axi_ar_if.avalid && axi_aw_if.avalid) begin
106             // Collision: Determine Priority
107             if (starve_cnt >= STARVE_LIMIT) begin
108                 // Force Switch (Starvation Avoidance)
109                 if (last_alloc_wr) ar_grant = 1'b1; // Was Writing,
                    force Read
110                 else
                    aw_grant = 1'b1; // Was Reading,
                    force Write
111             end else begin
112                 // Inertia (Follow last direction)
113                 if (last_alloc_wr) aw_grant = 1'b1;
114                 else
                    ar_grant = 1'b1;
115             end
116         end else if (axi_ar_if.avalid) begin
117             ar_grant = 1'b1;
118         end else if (axi_aw_if.avalid) begin
119             aw_grant = 1'b1;
120         end
121     end
122 end
123
124 assign axi_aw_if.aready = aw_grant;
125 assign axi_ar_if.aready = ar_grant;
126
127
```

# APPENDIX : ADDR Decoder Code

```
128 // Allocation Logic (Find first '0' in valid_vec)
129 always_comb begin
130     alloc_ptr = '0;
131     for(int k=0; k<ROB_DEPTH; k++) begin
132         if (!rob[k].valid) begin
133             alloc_ptr = k[$clog2(ROB_DEPTH)-1:0];
134             break;
135         end
136     end
137 end
138
139 // 3. Dispatch Logic (Arbitration & De-allocation)
140 // Issue Candidate Selection
141 logic [ROB_DEPTH-1:0] can_issue; // Bit mask of issuable slots
142 logic [ROB_DEPTH-1:0] issue_grant_oh; // One-hot grant
143 logic issue_valid;
144
145 // Bank Status
146 logic [*DRAM_BK_CNT-1:0] bank_ready;
147 generate
148     for(i=0; i<`DRAM_BK_CNT; i++) assign bank_ready[i] = req_if_arr
149     [i].ready;
150 endgenerate
```

```
151 // Ordering Check Logic (Hazard Detection)
152 logic [ROB_DEPTH-1:0] hazard_block;
153
154 always_comb begin
155     hazard_block = '0;
156
157     for(int k=0; k<ROB_DEPTH; k++) begin
158         if (rob[k].valid) begin
159             // Check against ALL other older valid slots
160             for(int j=0; j<ROB_DEPTH; j++) begin
161                 if (k != j && rob[j].valid) begin
162                     // Assuming ROB is NOT a circular queue but a
163                     // pool,
164                     // we need real timestamps. For simplicity here:
165                     // Compare seq_num: If J < K (J is older), J
166                     // must be checked.
167                     if (rob[j].seq_num < rob[k].seq_num) begin
168
169                         // 1. Same ID Hazard (Strict Ordering for
170                         // Same ID)
171                         if (rob[j].id == rob[k].id)
172                             hazard_block[k] = 1'b1;
173
174                         // 2. RAW / WAW / WAR Hazard (Address Match)
175                         // Even if ID is different, same address
176                         // access needs ordering
177                         // (Though usually AXI allows reordering if
178                         // ID diff,
179                         // DRAM coherence must be maintained)
180                         // simple row/bank check is not enough, need
181                         // full addr match
182                         // (Approximation: Check Bank & Row)
183                         if (rob[j].target_ba == rob[k].target_ba &&
184                             rob[j].target_ra == rob[k].target_ra &&
185                             rob[j].target_ca == rob[k].target_ca)
```



## APPENDIX : ADDR Decoder Code

```

180         begin // Simple Col check added
181             hazard_block[k] = 1'b1;
182         end
183     end
184 end
185
186 // Determine Candidates
187 always_comb begin
188     for(int k=0; k<ROB_DEPTH; k++) begin
189         can_issue[k] = 1'b0;
190         if (rob[k].valid) begin
191             // Condition 1: Target Bank is Ready
192             if (bank_ready[rob[k].target_ba]) begin
193                 // Condition 2: No Hazard blocking
194                 if (!hazard_block[k]) begin
195                     can_issue[k] = 1'b1;
196                 end
197             end
198         end
199     end
200 end
201
202 end
203
204

```

```

205 // Pick the "Best" Candidate
206 // Policy: Oldest First (to prevent starvation in Reorder Buffer)
207 // (Simply scan from seq_num perspective or simple iterate)
208 rob_entry_t winner;
209 logic [$clog2(ROB_DEPTH)-1:0] winner_idx;
210 logic found_winner;
211
212 always_comb begin
213     issue_grant_oh = '0;
214     winner_idx = '0;
215     found_winner = 1'b0;
216
217     // Simple Scan: Priority to "Smallest Seq Num" among candidates
218     // This is inefficient in logic, but logically correct.
219     // Optimization: Just pick First Valid found in linear scan?
220     // No, that biases towards low index slots.
221
222     // 1st Pass: Try to find READ in candidates
223     for(int k=0; k<ROB_DEPTH; k++) begin
224         if (can_issue[k] && !rob[k].wr) begin
225             // Ideally pick the oldest read, but picking ANY read
226             // is better effectively
227             issue_grant_oh[k] = 1'b1;
228             found_winner = 1'b1;
229             winner_idx = k[$clog2(ROB_DEPTH)-1:0]; // Cast
230             break;
231         end
232     end
233

```

## APPENDIX : ADDR Decoder Code

```
233 // 2nd Pass: If no read, pick WRITE
234 if (!found_winner) begin
235     for(int k=0; k<ROB_DEPTH; k++) begin
236         if (can_issue[k]) begin
237             issue_grant_oh[k] = 1'b1;
238             found_winner = 1'b1;
239             winner_idx = k[$clog2(ROB_DEPTH)-1:0]; // Cast
240             break;
241         end
242     end
243 end
244 end
```

```
246 // 4. Sequential Update
247 always_ff @(posedge clk or negedge rst_n) begin
248     if (!rst_n) begin
249         global_seq_num <= '0;
250         for(int k=0; k<ROB_DEPTH; k++) rob[k].valid <= 1'b0;
251     end else begin
252
253         // Allocation (Input)
254         if (ar_grant) begin // Read Accepted
255             rob[alloc_ptr].valid <= 1'b1;
256             rob[alloc_ptr].wr <= 1'b0;
257             rob[alloc_ptr].id <= axi_ar_if.aid;
258             rob[alloc_ptr].len <= axi_ar_if.alen;
259             rob[alloc_ptr].addr <= axi_ar_if.aaddr;
260             rob[alloc_ptr].seq_num <= global_seq_num;
261             rob[alloc_ptr].target_ba <= get_dram_ba(axi_ar_if.aaddr);
262             rob[alloc_ptr].target_ra <= get_dram_ra(axi_ar_if.aaddr);
263             rob[alloc_ptr].target_ca <= get_dram_ca(axi_ar_if.aaddr);
264
265             global_seq_num <= global_seq_num + 1;
266         end else if (aw_grant) begin // Write Accepted
267             rob[alloc_ptr].valid <= 1'b1;
268             rob[alloc_ptr].wr <= 1'b1;
269             rob[alloc_ptr].id <= axi_aw_if.aid;
270             rob[alloc_ptr].len <= axi_aw_if.alen;
271             rob[alloc_ptr].addr <= axi_aw_if.aaddr;
272             rob[alloc_ptr].seq_num <= global_seq_num;
273             rob[alloc_ptr].target_ba <= get_dram_ba(axi_aw_if.aaddr);
274             rob[alloc_ptr].target_ra <= get_dram_ra(axi_aw_if.aaddr);
275             rob[alloc_ptr].target_ca <= get_dram_ca(axi_aw_if.aaddr);
276
277             global_seq_num <= global_seq_num + 1;
278         end
279     end
280 end
```

## APPENDIX : ADDR Decoder Code

```
280 // Issue (Output) - Clear the slot
281 if (found_winner) begin
282     rob[winner_idx].valid <= 1'b0;
283
284     // If Alloc happened at same cycle on same slot
285     // (impossible due to logic, but for safety)
286     // If alloc_ptr == winner_idx, we would overwrite.
287     // However, current logic: IF alloc, we write '1'. IF
288     // issue, we write '0'.
289     // If both? Valid should stay '1' (New item replaces Old
290     // item).
291     // But alloc_ptr finds '0' valid. So alloc_ptr never
292     // points to a valid slot that is being issued?
293     // Actually, if we clear it NOW, it becomes free NEXT
294     // cycle. So no conflict.
295 end
296 end
297 end
```

```
294 // 5. Connect Output Interface
295 genvar geni;
296 generate
297     for (geni=0; geni<`DRAM_BK_CNT; geni=geni+1) begin
298         // If this bank is the target of the winner, assert valid
299         assign req_if_arr[geni].valid = found_winner && (rob
300             [winner_idx].target_ba == geni);
301
302         // Mux out the winner info
303         assign req_if_arr[geni].wr      = rob[winner_idx].wr;
304         assign req_if_arr[geni].id      = rob[winner_idx].id;
305         assign req_if_arr[geni].len     = rob[winner_idx].len;
306         assign req_if_arr[geni].seq_num = rob[winner_idx].seq_num;
307         assign req_if_arr[geni].ra      = rob[winner_idx].target_ra;
308         assign req_if_arr[geni].ca      = rob[winner_idx].target_ca;
309     end
310 endgenerate
311 endmodule // SAL_ADDR_DECODER_OO
```

# APPENDIX : Scheduler code

```
`include "TIME_SCALE.svh"
`include "SAL_DDR_PARAMS.svh"

// Force Sync Update 2
`define DRAM_BK_CNT2 4

module SAL_SCHED
#(
    parameter    bk_cnt = 4
)
(
    // clock & reset
    input         clk,
    input         rst_n,

    TIMING_IF.MON    timing_if,

    // requests from bank controllers
    BK_CTRL_IF.SCHED    bk_if,

    SCHED_IF.SCHED    sched_if
);

//Our code
// 1. Parameters & Localparams
localparam STARVATION_LIMIT = 8'd16; // Fixed at 16 per user requirement

// 2. Global Timing Counters

// 2.1 tRRD
wire is_zero_trrd;
SAL_TIMING_CNTR #(CNTR_WIDTH(`T_RRD_WIDTH)) u_cnt_trrd (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.act_gnt), .reset_value_i(timing_if.t_rrd_m1),
    .is_zero_o(is_zero_trrd));

// 2.2 tCCD
wire is_zero_tccd;
SAL_TIMING_CNTR #(CNTR_WIDTH(`T_CCD_WIDTH)) u_cnt_tccd (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.rd_gnt | sched_if.wr_gnt), .reset_value_i(timing_if.t_ccd_m1),
    .is_zero_o(is_zero_tccd));

// 2.3 tWTR
wire is_zero_twtw;
SAL_TIMING_CNTR #(CNTR_WIDTH(`T_WTR_WIDTH)) u_cnt_twtw (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.wr_gnt), .reset_value_i(timing_if.t_wtr_m1),
    .is_zero_o(is_zero_twtw));

// 2.4 tRTW
wire is_zero_trtw;
SAL_TIMING_CNTR #(CNTR_WIDTH(`T_RTW_WIDTH)) u_cnt_trtw (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.rd_gnt), .reset_value_i(timing_if.t_rtw_m1),
    .is_zero_o(is_zero_trtw));
```

```
// 2.5 tRFC
wire is_zero_trfc;
SAL_TIMING_CNTR #(CNTR_WIDTH(`T_RFC_WIDTH)) u_cnt_trfc (
    .clk(clk), .rst_n(rst_n), .reset_cmd_i(sched_if.ref_gnt), .reset_value_i(timing_if.t_rfc_m1),
    .is_zero_o(is_zero_trfc));

// 3. Optimization Logic
// 3.1 Last Operation Direction

logic last_op_was_write;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        last_op_was_write <= 1'b0;
    end else begin
        if (sched_if.wr_gnt) begin
            last_op_was_write <= 1'b1;
        end else if (sched_if.rd_gnt) begin
            last_op_was_write <= 1'b0;
        end
        // Keep state on NOP or ACT/PRE
    end
end

// 3.2 Starvation Protection (Safety Net)
// Counts how long a valid request has been waiting without a grant.
logic [7:0] wait_timer [bk_cnt];
logic [bk_cnt-1:0] is_starved;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for(int i=0; i<bk_cnt; i++) wait_timer[i] <= '0;
    end else begin
        for(int i=0; i<bk_cnt; i++) begin
            // If request exists...
            if (bk_if.reqs[i].rd_req || bk_if.reqs[i].wr_req || bk_if.reqs[i].act_req) begin
                // ...and NOT granted yet
                if (!bk_if.gnts[i].rd_gnt && !bk_if.gnts[i].wr_gnt && !bk_if.gnts[i].act_gnt) begin
                    // This prevents banks from being blocked indefinitely by a stream of R/W in other banks.
                    if (wait_timer[i] < STARVATION_LIMIT)
                        wait_timer[i] <= wait_timer[i] + 1;
                end else begin
                    wait_timer[i] <= '0; // Granted! Reset timer.
                end
            end else begin
                wait_timer[i] <= '0; // No request. Reset timer.
            end
        end
    end
end

assign is_starved[0] = (wait_timer[0] == STARVATION_LIMIT);
assign is_starved[1] = (wait_timer[1] == STARVATION_LIMIT);
assign is_starved[2] = (wait_timer[2] == STARVATION_LIMIT);
assign is_starved[3] = (wait_timer[3] == STARVATION_LIMIT);
```

# APPENDIX : Scheduler code

```
// 4. Arbitration Logic

// Request Vectors
logic [bk_cnt-1:0] req_acts, req_rds, req_wrs, req_pres;
genvar i;
generate
    for(i=0; i<bk_cnt; i++) begin
        assign req_acts[i] = bk_if.reqs[i].act_req;
        assign req_rds[i] = bk_if.reqs[i].rd_req;
        assign req_wrs[i] = bk_if.reqs[i].wr_req;
        assign req_pres[i] = bk_if.reqs[i].pre_req;
    end
endgenerate

logic [bk_cnt-1:0] cand_valid, cand_timing_ok, base_set;
logic [bk_cnt-1:0] cand_emergency, cand_prio_dir, cand_row_hit;
logic [bk_cnt-1:0] final_candidates;
logic [bk_cnt-1:0] winner_oh;

always_comb begin
    // A. Validity Check
    cand_valid = req_acts | req_rds | req_wrs | req_pres;

    // B. Timing Check
    cand_timing_ok = '0;
    if (is_zero_trfc) begin
        for(int k=0; k<bk_cnt; k++) begin
            if (req_acts[k]) cand_timing_ok[k] = is_zero_trrd;
            else if (req_rds[k]) cand_timing_ok[k] = is_zero_tccd & is_zero_twtw;
            else if (req_wrs[k]) cand_timing_ok[k] = is_zero_tccd & is_zero_trtw;
            else if (req_pres[k]) cand_timing_ok[k] = '1'b1;
        end
    end

    // ID-Specific Ordering Check
    logic [bk_cnt-1:0] id_ordering_safe;

    // Check if any other bank has the SAME ID but a SMALLER seq_num (Older request)
    always_comb begin
        id_ordering_safe = '1; // Default to safe (allow all)

        for(int k=0; k<bk_cnt; k++) begin
            // For a candidate bank K
            if (bk_if.reqs[k].act_req || bk_if.reqs[k].rd_req || bk_if.reqs[k].wr_req || bk_if.reqs[k].pre_req)
                begin
                    for(int j=0; j<bk_cnt; j++) begin
                        if (k != j) begin
                            // If bank J also has a request
                            if (bk_if.reqs[j].act_req || bk_if.reqs[j].rd_req || bk_if.reqs[j].wr_req || bk_if.reqs[j].pre_req) begin
                                // And IDs match
                                if (bk_if.reqs[k].id == bk_if.reqs[j].id) begin
                                    // Wraparound-safe Comparison:
                                    // Check if J < K in a circular buffer sense.
                                    logic [7:0] diff;
                                end
                            end
                        end
                    end
                end
        end
    end
end
```

```
diff = bk_if.reqs[k].seq_num - bk_if.reqs[j].seq_num;
if (diff < 8'd128 && diff != 0) begin
    id_ordering_safe[k] = '1'b0; // K must wait for J
end
end
end
end
end
end

always_comb begin
    // Base Set: Valid + Timing OK + ID Safe
    base_set = cand_valid & cand_timing_ok & id_ordering_safe;

    // C. Intelligent Filtering (Priority System)

    // Priority 0: Starvation (Emergency)
    // If anyone has waited too long, they bypass optimization rules.
    cand_emergency = base_set & is_starved;

    // Priority 1: Same Direction (Maximize Bus Utilization)
    // Try to keep doing what we were doing (Read or Write) to avoid tWTR/tRTW bubbles.
    if (last_op_was_write)
        cand_prio_dir = base_set & req_wrs;
    else
        cand_prio_dir = base_set & req_rds;

    // Priority 2: Row Hits (Data Transfer > ACT/PRE)
    // If we can't maintain direction, at least try to do SOME data transfer.
    cand_row_hit = base_set & (req_rds | req_wrs);

    // Priority 4: Wait more if Precharge is issued
    // To avoid PRE -> ACT/RD/WR violation immediately

    // Final Selection Hierarchy
    if (!cand_emergency) final_candidates = cand_emergency; // Emergency First!
    else if (!cand_prio_dir) final_candidates = cand_prio_dir; // Same Direction Second
    else if (!cand_row_hit) final_candidates = cand_row_hit; // Then Any Hit (Data Bus Open)
    else final_candidates = base_set & ~req_rds & ~req_wrs; // Avoid RD/WR if not hit to prevent blind access
end

// D. Global Sequence Number Based Grant (Oldest First)
// Instead of Round Robin, we pick the candidate with the 'smallest' seq_num.
// Smallest means oldest in a circular buffer sense.

always_comb begin
    winner_oh = '0;
end
```

# APPENDIX : Scheduler code

```
// Only if there are candidates
if (!final_candidates) begin
    logic [7:0] min_seq;
    int winner_idx;
    logic first_found;

    winner_idx = 0;
    first_found = 1'b0;
    min_seq = '0; // default (not actually used since first_found logic overwrites it)

    for (int k=0; k<bk_cnt; k++) begin
        if (final_candidates[k]) begin
            if (!first_found) begin
                // First valid candidate we see becomes the temporary winner
                winner_idx = k;
                // Use a dummy variable or specific logic to get min_seq
                min_seq = bk_if.reqs[k].seq_num;
                first_found = 1'b1;
            end else begin
                // We compare current k's seq (seq_k) vs min_seq
                // We want to check: Is seq_k < min_seq (older)?
                // Logic: wrapped diff (seq_k - min_seq)
                // If result is small pos -> seq_k > min_seq (newer)
                // If result is large pos (neg) -> seq_k < min_seq (older)

                logic [7:0] seq_k;
                logic [7:0] diff_k_min;

                seq_k = bk_if.reqs[k].seq_num;
                diff_k_min = seq_k - min_seq;

                // If diff_k_min >= 128, it means seq_k is effectively smaller (older) due to wrap-around
                // or just simple smaller number.
                // Example: seq_k=255, min=0. diff = 255. 255 >= 128. True. k is older.
                // Example: seq_k=0, min=255. diff = 1. False. k is newer.

                if (diff_k_min >= 8'd128) begin
                    winner_idx = k;
                    min_seq = seq_k;
                end
            end
        end
    end

    winner_oh[winner_idx] = 1'b1;
end

// 5. Output Grant Generation
logic [bk_cnt-1:0] all_ref_reqs;
genvar k;
generate
    for (k=0; k<bk_cnt; k++) begin
        assign bk_if.gnts[k].act_gnt = winner_oh[k] && req_acts[k];
        assign bk_if.gnts[k].rd_gnt = winner_oh[k] && req_rds[k];
        assign bk_if.gnts[k].wr_gnt = winner_oh[k] && req_wrs[k];
    end
endgenerate
```

```
assign bk_if.gnts[k].pre_gnt = winner_oh[k] && req_pres[k];

assign all_ref_reqs[k] = bk_if.reqs[k].ref_req;
// Grant refresh to all banks when the scheduler decides to refresh
assign bk_if.gnts[k].ref_gnt = sched_if.ref_gnt;
end

endgenerate

assign sched_if.act_gnt = |(winner_oh & req_acts);
assign sched_if.rd_gnt = |(winner_oh & req_rds);
assign sched_if.wr_gnt = |(winner_oh & req_wrs);
assign sched_if.ref_gnt = (!is_zero_trfc) ? 1'b0 : (all_ref_reqs);
assign sched_if.pre_gnt = |(winner_oh & req_pres);

// 6. Data Muxing
always_comb begin
    sched_if.ba = '0;
    sched_if.ra = '0;
    sched_if.ca = '0;
    sched_if.id = '0;
    sched_if.len = '0;

    for (int k=0; k<bk_cnt; k++) begin
        if (winner_oh[k]) begin
            sched_if.ba = bk_if.reqs[k].ba;
            sched_if.ra = bk_if.reqs[k].ra;
            sched_if.ca = bk_if.reqs[k].ca;
            sched_if.id = bk_if.reqs[k].id;
            sched_if.len = bk_if.reqs[k].len;
        end
    end
end

endmodule
```

# APPENDIX : Gate-Level Implementation - Bank Controller Code

```
1  `include "TIME_SCALE.svh"
2  `include "SAL_DDR_PARAMS.svh"
3
4  module SAL_BK_CTRL
5  #(
6      parameter BK_ID = 0
7  )
8  (
9      input wire          clk,
10     input wire          rst_n,
11     TIMING_IF.MON       timing_if,
12     REQ_IF.DST          req_if,
13     output bk_req_t      bk_reqs,
14     input bk_gnt_t       bk_gnts,
15     input wire          ref_req_i,
16     output wire          ref_gnt_o
17 );
18
19 // 0. Constant Generation (Logic 0 & Logic 1)
20 wire logic_0, logic_1;
21 AND2 u_tie_low (.A1(1'b0), .A2(1'b0), .Y(logic_0));
22 INV  u_tie_high (.A(logic_0), .Y(logic_1));
23
24 // 1. Internal Wires Definition
25 wire [6:0] state;      // Current State (Q)
26 wire [6:0] state_n;    // Next State (D)
27
28 // Data Path Registers
29 wire req_buf_valid;
30 wire req_buf_valid_n;
31 wire req_buf_wr;
32 wire [`AXI_ID_WIDTH-1:0] req_buf_id;
33 wire [`AXI_LEN_WIDTH-1:0] req_buf_len;
34 wire [`DRAM_RA_WIDTH-1:0] req_buf_ra;
35 wire [`DRAM_RA_WIDTH-1:0] cur_ra;
```

```
36 wire [`DRAM_CA_WIDTH-1:0] req_buf_ca;
37 wire [2:0] req_buf_seq_num;
38
39 // Timing Status Signals
40 wire is_t_rc_met, is_t_rcd_met, is_t_rp_met, is_t_ras_met,
41     is_t_rfc_met;
42 wire is_t_rtp_met, is_t_wtp_met, is_row_open_met, is_t_wtr_met,
43     is_t_rtw_met;
44
45 // Logic Flags
46 wire ref_req_pending;
47 wire latch_enable;
48
49 // 2. Timing Counters Instantiation
50
51 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_RC_WIDTH)) u_rc_cnt(.clk
52     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i
53     (timing_if.t_rc_m1),.is_zero_o(is_t_rc_met));
54 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_RCD_WIDTH)) u_rcd_cnt(.clk
55     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i
56     (timing_if.t_rcd_m2),.is_zero_o(is_t_rcd_met));
57 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_RP_WIDTH)) u_rp_cnt(.clk
58     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.pre_gnt),.reset_value_i
59     (timing_if.t_rp_m2),.is_zero_o(is_t_rp_met));
60 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_RAS_WIDTH)) u_ras_cnt(.clk
61     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i
62     (timing_if.t_ras_m1),.is_zero_o(is_t_ras_met));
63 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_RTP_WIDTH)) u_rtp_cnt(.clk
64     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.rd_gnt),.reset_value_i
65     (timing_if.t_rtp_m1),.is_zero_o(is_t_rtp_met));
66 SAL_TIMING_CNTR #(C.NTR_WIDTH(`T_WTP_WIDTH)) u_wtp_cnt(.clk
67     (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.wr_gnt),.reset_value_i
68     (timing_if.t_wtp_m1),.is_zero_o(is_t_wtp_met));
```

# APPENDIX : Gate-Level Implementation - Bank Controller Code

```
55 SAL_TIMING_CNTR #(.CNTR_WIDTH(`ROW_OPEN_WIDTH)) u_row_open_cnt(.clk
    (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.act_gnt),.reset_value_i
    (timing_if.row_open_cnt),.is_zero_o(is_row_open_met));
56 SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RFC_WIDTH)) u_rfc_cnt(.clk
    (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.ref_gnt),.reset_value_i
    (timing_if.t_rfc_m2),.is_zero_o(is_t_rfc_met));
57 SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_WTR_WIDTH)) u_wtr_cnt(.clk
    (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.wr_gnt),.reset_value_i
    (timing_if.t_wtr_m1),.is_zero_o(is_t_wtr_met));
58 SAL_TIMING_CNTR #(.CNTR_WIDTH(`T_RTW_WIDTH)) u_rtw_cnt(.clk
    (clk),.rst_n(rst_n),.reset_cmd_i(bk_gnts.rd_gnt),.reset_value_i
    (timing_if.t_rtw_m1),.is_zero_o(is_t_rtw_met));
59
60 // 3. Periodic Refresh Logic
61 wire n_clear_cond, set_or_hold, next_pending;
62 NAND2 u_nand_clear (.A1(bk_reqs.ref_req), .A2(bk_gnts.ref_gnt), .Y
    (n_clear_cond));
63 OR2 u_or_set_hold (.A1(ref_req_i), .A2(ref_req_pending), .Y
    (set_or_hold));
64 AND2 u_and_next (.A1(n_clear_cond), .A2(set_or_hold), .Y
    (next_pending));
65 DFF u_dff_pending (.D(next_pending), .RST_n(rst_n), .CLK(clk), .Q
    (ref_req_pending), .QN());
66
67 // 4. Data Path Registers & Control
68
69 // 4.1 Latch Enable & Ready Signal
70 wire n_req_buf_valid;
71 INV u_inv_valid (.A(req_buf_valid), .Y(n_req_buf_valid));
72 INV u_drive_rdy (.A(req_buf_valid), .Y(req_if.ready));
73 AND2 u_latch_en (.A1(req_if.valid), .A2(n_req_buf_valid), .Y
    (latch_enable));
```

```
74
75 // 4.2 Payload Registers (Using Generate + MUX + DFF)
76 genvar i;
77 generate
78     wire wr_next;
79     MUX21 u_mux_wr (.A1(req_if.wr), .A2(req_buf_wr), .S0
    (latch_enable), .Y(wr_next));
80     DFF u_dff_wr (.D(wr_next), .RST_n(rst_n), .CLK(clk), .Q
    (req_buf_wr), .QN());
81
82     for(i=0; i<`AXI_ID_WIDTH; i=i+1) begin : gen_buf_id
83         wire id_next;
84         MUX21 u_mux_id (.A1(req_if.id[i]), .A2(req_buf_id[i]), .S0
    (latch_enable), .Y(id_next));
85         DFF u_dff_id (.D(id_next), .RST_n(rst_n), .CLK(clk), .Q
    (req_buf_id[i]), .QN());
86     end
87
88     for(i=0; i<`AXI_LEN_WIDTH; i=i+1) begin : gen_buf_len
89         wire len_next;
90         MUX21 u_mux_len (.A1(req_if.len[i]), .A2(req_buf_len[i]), .S0
    (latch_enable), .Y(len_next));
91         DFF u_dff_len (.D(len_next), .RST_n(rst_n), .CLK(clk), .Q
    (req_buf_len[i]), .QN());
92     end
93
94     for(i=0; i<`DRAM_RA_WIDTH; i=i+1) begin : gen_buf_ra
95         wire ra_next;
96         MUX21 u_mux_ra (.A1(req_if.ra[i]), .A2(req_buf_ra[i]), .S0
    (latch_enable), .Y(ra_next));
97         DFF u_dff_ra (.D(ra_next), .RST_n(rst_n), .CLK(clk), .Q
    (req_buf_ra[i]), .QN());
98     end
```



# APPENDIX : Gate-Level Implementation - Bank Controller Code

```

99
100 ✓ for(i=0; i<`DRAM_CA_WIDTH; i=i+1) begin : gen_buf_ca
101     wire ca_next;
102     MUX21 u_mux_ca (.A1(req_if.ca[i]), .A2(req_buf_ca[i]), .S0
        (latch_enable), .Y(ca_next));
103     DFF u_dff_ca (.D(ca_next), .RST_n(rst_n), .CLK(clk), .Q
        (req_buf_ca[i]), .QN());
104 end
105
106 ✓ for(i=0; i<3; i=i+1) begin : gen_buf_seq
107     wire seq_next;
108     MUX21 u_mux_seq (.A1(req_if.seq_num[i]), .A2(req_buf_seq_num
        [i]), .S0(latch_enable), .Y(seq_next));
109     DFF u_dff_seq (.D(seq_next), .RST_n(rst_n), .CLK(clk), .Q
        (req_buf_seq_num[i]), .QN());
110 end
111
112 ✓ for(i=0; i<`DRAM_RA_WIDTH; i=i+1) begin : gen_cur_ra
113     wire cur_next_val;
114     MUX21 u_mux_cur (.A1(req_buf_ra[i]), .A2(cur_ra[i]), .S0
        (bk_gnts.act_gnt), .Y(cur_next_val));
115     DFF u_dff_cur_ra (.D(cur_next_val), .RST_n(rst_n), .CLK
        (clk), .Q(cur_ra[i]), .QN());
116 end
117 endgenerate
118
119 // 4.3 Request Buffer Valid Register Logic
120 wire valid_set, valid_clear, n_valid_clear, valid_hold;
121
122 AND2 u_v_set (.A1(req_if.valid), .A2(n_req_buf_valid), .Y
        (valid_set));
123 OR2 u_v_clr (.A1(bk_gnts.wr_gnt), .A2(bk_gnts.rd_gnt), .Y
        (valid_clear));

```

```

124 INV u_inv_v_clr (.A(valid_clear), .Y(n_valid_clear));
125 AND2 u_v_hold (.A1(req_buf_valid), .A2(n_valid_clear), .Y
        (valid_hold));
126 OR2 u_v_next (.A1(valid_hold), .A2(valid_set), .Y(req_buf_valid_n));
127
128 DFF u_dff_req_valid (.D(req_buf_valid_n), .RST_n(rst_n), .CLK
        (clk), .Q(req_buf_valid), .QN());
129
130 // 5. Combinational Logic
131
132 // 5.1 Row Miss Detector
133 wire [`DRAM_RA_WIDTH-1:0] ra_diff_bits;
134 ✓ generate
135 ✓ for(i=0; i<`DRAM_RA_WIDTH; i=i+1) begin : ra_comp
136     XOR2 u_xor_ra (.A1(req_buf_ra[i]), .A2(cur_ra[i]), .Y
        (ra_diff_bits[i]));
137 end
138 endgenerate
139
140 wire [`DRAM_RA_WIDTH-1:0] chain_w;
141 wire row_miss;
142
143 ✓ generate
144     AND2 u_base_inv (.A1(ra_diff_bits[0]), .A2(logic_1), .Y(chain_w
        [0]));
145     genvar m;
146 ✓ for(m=1; m<`DRAM_RA_WIDTH; m=m+1) begin : gen_or_chain
147         OR2 u_miss_or (.A1(chain_w[m-1]), .A2(ra_diff_bits[m]), .Y
            (chain_w[m]));
148     end
149     AND2 u_out_inv (.A1(chain_w[`DRAM_RA_WIDTH-1]), .A2(logic_1), .Y
        (row_miss));
150 endgenerate
151

```

# APPENDIX : Gate-Level Implementation - Bank Controller Code

```
152 // 5.2 Priority Logic Signals
153
154 // 1. Refresh Priority Logic
155 wire cond_ref_needed, timer_ref_ready;
156 AND3 u_tm_ref (.A1(is_t_wtp_met), .A2(is_t_rtp_met), .A3
157 (is_t_ras_met), .Y(timer_ref_ready));
158 AND2 u_cond_ref (.A1(ref_req_pending), .A2(timer_ref_ready), .Y
159 (cond_ref_needed));
160
161 // 2. Row Miss Logic (Timing Aware)
162 wire n_cond_ref, miss_timers, raw_miss_req, cond_miss_needed;
163 INV u_inv_ref (.A(cond_ref_needed), .Y(n_cond_ref));
164 AND3 u_tm_miss (.A1(is_t_ras_met), .A2(is_t_wtp_met), .A3
165 (is_t_rtp_met), .Y(miss_timers));
166 AND3 u_raw_miss (.A1(req_buf_valid), .A2(row_miss), .A3
167 (miss_timers), .Y(raw_miss_req));
168 AND2 u_prio_miss (.A1(raw_miss_req), .A2(n_cond_ref), .Y
169 (cond_miss_needed));
170 wire n_pure_row_miss;
171 INV u_inv_pure_miss (.A(row_miss), .Y(n_pure_row_miss));
172
173 // 3. Write Priority Logic
174 wire n_cond_miss_NOT_USED, wr_timers, raw_wr_req, cond_wr_needed;
175 INV u_inv_miss (.A(cond_miss_needed), .Y(n_cond_miss_NOT_USED));
176 AND2 u_tm_wr (.A1(is_t_rcd_met), .A2(is_t_rtw_met), .Y(wr_timers));
177 AND3 u_raw_wr (.A1(req_buf_valid), .A2(req_buf_wr), .A3(wr_timers), .Y
178 (raw_wr_req));
179 AND3 u_prio_wr (.A1(raw_wr_req), .A2(n_cond_ref), .A3
180 (n_pure_row_miss), .Y(cond_wr_needed));
181
182 // 4. Read Priority Logic
183 wire n_cond_wr, n_req_buf_wr, rd_timers, raw_rd_req, cond_rd_needed;
184 INV u_inv_wr_prio (.A(cond_wr_needed), .Y(n_cond_wr));
185 INV u_inv_wr_flag (.A(req_buf_wr), .Y(n_req_buf_wr));
186 AND2 u_tm_rd (.A1(is_t_rcd_met), .A2(is_t_wtr_met), .Y(rd_timers));
```

```
180 AND3 u_raw_rd (.A1(req_buf_valid), .A2(n_req_buf_wr), .A3
181 (rd_timers), .Y(raw_rd_req));
182 AND3 u_prio_rd (.A1(raw_rd_req), .A2(n_pure_row_miss), .A3
183 (n_cond_ref), .Y(cond_rd_needed));
184
185 // 5.3 Output Driving
186
187 // (a) Drive bk_reqs.ba
188 genvar j;
189 generate
190     for(j=0; j<2; j=j+1) begin : gen_ba_mux
191         MUX21 u_mux_ba (
192             .A1(logic_1),
193             .A2(logic_0),
194             .S0(BK_ID[j]),
195             .Y(bk_reqs.ba[j])
196         );
197     end
198 endgenerate
199
200 // (b) Drive bk_reqs.ref_req
201 wire idle_timer_ref, idle_ref_req;
202 AND2 u_id_tm (.A1(is_t_rp_met), .A2(is_t_rc_met), .Y
203 (idle_timer_ref));
204 AND3 u_out_ref_idle (.A1(state[0]), .A2(ref_req_pending), .A3
205 (idle_timer_ref), .Y(idle_ref_req));
206 AND2 u_buf_idle (.A1(idle_ref_req), .A2(logic_1), .Y(bk_reqs.
207 ref_req));
208
209 // (c) Drive bk_reqs.act_req
210 wire n_idle_ref_req, idle_timer_act, raw_act_req, act_req_internal;
211 INV u_inv_id_ref (.A(idle_ref_req), .Y(n_idle_ref_req));
212 AND2 u_id_tm_act (.A1(is_t_rc_met), .A2(is_t_rp_met), .Y
213 (idle_timer_act));
```

# APPENDIX : Gate-Level Implementation - Bank Controller Code

```
209 AND3 u_raw_act (.A1(state[0]), .A2(req_buf_valid), .A3
    (idle_timer_act), .Y(raw_act_req));
210 AND2 u_out_act (.A1(raw_act_req), .A2(n_idle_ref_req), .Y
    (act_req_internal));
211 AND2 u_buf_act (.A1(act_req_internal), .A2(logic_1), .Y(bk_reqs.
    act_req));
212
213 // (d) Drive bk_reqs.pre_req
214 wire active_pre_req, pre_req_internal;
215 OR2 u_or_pre_cond (.A1(cond_ref_needed), .A2(cond_miss_needed), .Y
    (active_pre_req));
216 AND2 u_out_pre (.A1(state[2]), .A2(active_pre_req), .Y
    (pre_req_internal));
217 AND2 u_buf_pre (.A1(pre_req_internal), .A2(logic_1), .Y(bk_reqs.
    pre_req));
218
219 // (e) Drive bk_reqs.wr_req
220 wire wr_req_internal, n_out_wr;
221 AND2 u_out_wr (.A1(state[2]), .A2(cond_wr_needed), .Y
    (wr_req_internal));
222 AND2 u_buf_wr (.A1(wr_req_internal), .A2(logic_1), .Y(bk_reqs.
    wr_req));
223
224
225 // (f) Drive bk_reqs.rd_req
226 wire rd_req_internal;
227 AND2 u_out_rd (.A1(state[2]), .A2(cond_rd_needed), .Y
    (rd_req_internal));
228 AND2 u_buf_rd (.A1(rd_req_internal), .A2(logic_1), .Y(bk_reqs.
    rd_req));
229
230
231 // (g) Drive ref_gnt_o
232 INV u_buf_gnt2 (.A(logic_1), .Y(ref_gnt_o));
```

```
233
234 // (h) Pass-through Data Bus
235 generate
236     for(i=0; i<`DRAM_RA_WIDTH; i=i+1) begin: pass_ra
237         AND2 u_and (.A1(req_buf_ra[i]), .A2(logic_1), .Y(bk_reqs.ra
            [i]));
238     end
239     for(i=0; i<`DRAM_CA_WIDTH; i=i+1) begin: pass_ca
240         AND2 u_and (.A1(req_buf_ca[i]), .A2(logic_1), .Y(bk_reqs.ca
            [i]));
241     end
242     for(i=0; i<`AXI_ID_WIDTH; i=i+1) begin: pass_id
243         AND2 u_and (.A1(req_buf_id[i]), .A2(logic_1), .Y(bk_reqs.id
            [i]));
244     end
245     for(i=0; i<`AXI_LEN_WIDTH; i=i+1) begin: pass_len
246         AND2 u_and (.A1(req_buf_len[i]), .A2(logic_1), .Y(bk_reqs.len
            [i]));
247     end
248     for(i=0; i<3; i=i+1) begin: pass_seq
249         AND2 u_and (.A1(req_buf_seq_num[i]), .A2(logic_1), .Y
            (bk_reqs.seq_num[i]));
250     end
251 endgenerate
252
253 // 5.4 Next State Logic
254
255 // IDLE (state[0])
256 wire trans_pre_idle;
257 AND2 u_tr_pi (.A1(state[5]), .A2(is_t_rp_met), .Y(trans_pre_idle));
258 wire trans_ref_idle;
259 AND2 u_tr_ri (.A1(state[6]), .A2(is_t_rfc_met), .Y(trans_ref_idle));
260 wire idle_gnt_ref, idle_gnt_act;
261 AND2 u_ig_ref (.A1(idle_ref_req), .A2(bk_gnts.ref_gnt), .Y
    (idle_gnt_ref));
```

# APPENDIX : Gate-Level Implementation - Bank Controller Code

```
262 AND2 u_ig_act (.A1(act_req_internal), .A2(bk_gnts.act_gnt), .Y
    (idle_gnt_act));
263 wire idle_exit, n_idle_exit, hold_idle, next_idle_or;
264 OR2 u_id_exit (.A1(idle_gnt_ref), .A2(idle_gnt_act), .Y(idle_exit));
265 INV u_inv_ie (.A(idle_exit), .Y(n_idle_exit));
266 AND2 u_h_idle (.A1(state[0]), .A2(n_idle_exit), .Y(hold_idle));
267 OR2 u_ni_1 (.A1(trans_pre_idle), .A2(trans_ref_idle), .Y
    (next_idle_or));
268 OR2 u_ni_2 (.A1(next_idle_or), .A2(hold_idle), .Y(state_n[0]));
269
270 // ACTIVATING (state[1])
271 wire trans_idle_act;
272 AND2 u_trans_idle_act (.A1(idle_gnt_act), .A2(logic_1), .Y
    (trans_idle_act));
273 wire n_rcd_met, hold_act;
274 INV u_inv_rcd (.A(is_t_rcd_met), .Y(n_rcd_met));
275 AND2 u_h_act (.A1(state[1]), .A2(n_rcd_met), .Y(hold_act));
276 OR2 u_ns_1 (.A1(trans_idle_act), .A2(hold_act), .Y(state_n[1]));
277
278 // BANK_ACTIVE (state[2])
279 wire trans_act_active;
280 AND2 u_tr_aa (.A1(state[1]), .A2(is_t_rcd_met), .Y
    (trans_act_active));
281 wire active_gnt_pre, active_gnt_wr, active_gnt_rd;
282 AND2 u_ag_pre (.A1(pre_req_internal), .A2(bk_gnts.pre_gnt), .Y
    (active_gnt_pre));
283 AND2 u_ag_wr (.A1(wr_req_internal), .A2(bk_gnts.wr_gnt), .Y
    (active_gnt_wr));
284 AND2 u_ag_rd (.A1(rd_req_internal), .A2(bk_gnts.rd_gnt), .Y
    (active_gnt_rd));
285 wire active_exit_1, active_exit, n_active_exit, hold_active;
286 OR2 u_ae_1 (.A1(active_gnt_pre), .A2(active_gnt_wr), .Y
    (active_exit_1));
287 OR2 u_ae_2 (.A1(active_exit_1), .A2(active_gnt_rd), .Y
    (active_exit));
```

```
288 INV u_inv_ae (.A(active_exit), .Y(n_active_exit));
289 AND2 u_h_active (.A1(state[2]), .A2(n_active_exit), .Y(hold_active));
290 wire ns_2_or1, ns_2_or2;
291 OR2 u_ns2_1 (.A1(trans_act_active), .A2(state[3]), .Y(ns_2_or1));
292 OR2 u_ns2_2 (.A1(state[4]), .A2(hold_active), .Y(ns_2_or2));
293 OR2 u_ns2_f (.A1(ns_2_or1), .A2(ns_2_or2), .Y(state_n[2]));
294
295 // READING (state[3])
296 AND2 u_sn3 (.A1(active_gnt_rd), .A2(logic_1), .Y(state_n[3]));
297
298 // WRITING (state[4])
299 AND2 u_sn4 (.A1(active_gnt_wr), .A2(logic_1), .Y(state_n[4]));
300
301 // PRECHARGING (state[5])
302 wire n_rp_met, hold_pre;
303 INV u_inv_rp (.A(is_t_rp_met), .Y(n_rp_met));
304 AND2 u_h_pre (.A1(state[5]), .A2(n_rp_met), .Y(hold_pre));
305 OR2 u_ns_5 (.A1(active_gnt_pre), .A2(hold_pre), .Y(state_n[5]));
306
307 // REFRESH (state[6])
308 wire n_rfc_met, hold_ref;
309 INV u_inv_rfc (.A(is_t_rfc_met), .Y(n_rfc_met));
310 AND2 u_h_ref (.A1(state[6]), .A2(n_rfc_met), .Y(hold_ref));
311 OR2 u_ns_6 (.A1(idle_gnt_ref), .A2(hold_ref), .Y(state_n[6]));
312
```

## APPENDIX : Gate-Level Implementation - Bank Controller Code

```
313 // 6. FSM State Registers
314
315 // 6.1 State[0] (IDLE): 리셋 시 1
316 wire d_idle_inverted;
317 wire q_idle_internal;
318
319 INV u_inv_idle_in (.A(state_n[0]), .Y(d_idle_inverted));
320 DFF u_dff_state_idle (
321     .D(d_idle_inverted),
322     .RST_n(rst_n),
323     .CLK(clk),
324     .Q(q_idle_internal),
325     .QN()
326 );
327 INV u_inv_idle_out (.A(q_idle_internal), .Y(state[0]));
328
329 // 6.2 State[6:1]: 리셋 시 0
330 genvar k;
331 generate
332     for(k=1; k<7; k=k+1) begin : gen_state_reg_others
333         DFF u_dff_state (
334             .D(state_n[k]),
335             .RST_n(rst_n),
336             .CLK(clk),
337             .Q(state[k]),
338             .QN()
339         );
340     end
341 endgenerate
342
343 endmodule
```

# APPENDIX : Testbench Result (SAL\_TB\_TOP)

## 1 기본 Decoder & Round-Robin Scheduler(Baseline)

```
Succeed!  
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1338750.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1338750 ps
```

## 2 기본 Decoder & 설계한 Scheduler

```
Succeed!  
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1336250.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1336250 ps
```

## 3 설계한 Decoder & Round-Robin Scheduler(Baseline)

```
Succeed!  
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1358750.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1358750 ps
```

## 4 설계한 Decoder & 설계한 Scheduler

```
Succeed!  
$finish called from file "/home/urp01/chang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 178.  
$finish at simulation time 1356250.0 ps  
V C S   S i m u l a t i o n   R e p o r t  
Time: 1356250 ps
```

# APPENDIX : Testbench Result (SAL\_TB\_TOP\_REF)

## 1 기본 Decoder & Round-Robin Scheduler (Baseline)

```
Time 811000.0 ps: Refresh Command Detected!
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
```

## 2 기본 Decoder & 설계한 Scheduler

```
Time 811000.0 ps: Refresh Command Detected!
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
```

# APPENDIX : Testbench Result (SAL\_TB\_TOP\_REF)

## 3 설계한 Decoder & Round-Robin Scheduler(Baseline)

```
Time 811000.0 ps: Refresh Command Detected!
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
```

## 4 설계한 Decoder & 설계한 Scheduler

```
Time 811000.0 ps: Refresh Command Detected!
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 811250.0 ps INFO: Refresh
SAL_TB_TOP_REF.u_rank0.u_dram0.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram1.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram2.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram3.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram4.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram5.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram6.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
SAL_TB_TOP_REF.u_rank0.u_dram7.cmd_task: at time 916250.0 ps INFO: Activate bank 1 row 0000
```



# APPENDIX : Testbench Result (SAL\_TB\_TOP\_rep2)

## 1 기본 Decoder & Round-Robin Scheduler(Baseline)

```
[Result D] Duration: 195.000000 ps
```

```
[Result E] Duration: 427.500000 ps
```

```
=====
Test Completed.
=====
```

```
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 231.
```

```
$finish at simulation time 1771250.0 ps
```

```
V C S   S i m u l a t i o n   R e p o r t
```

```
Time: 1771250 ps
```

## 2 기본 Decoder & 설계한 Scheduler

```
[Result D] Duration: 197.500000 ps
```

```
[Result E] Duration: 427.500000 ps
```

```
=====
Test Completed.
=====
```

```
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 231.
```

```
$finish at simulation time 1773750.0 ps
```

```
V C S   S i m u l a t i o n   R e p o r t
```

```
Time: 1773750 ps
```

## APPENDIX : Testbench Result (SAL\_TB\_TOP\_rep2)

### 3 설계한 Decoder & Round-Robin Scheduler(Baseline)

```
[Result D] Duration: 140.000000 ps
```

```
[Result E] Duration: 437.500000 ps
```

```
=====
Test Completed.
```

```
=====
$finish called from file "/home/urp01/team_jang/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 231.
```

```
$finish at simulation time 1723750.0 ps
```

```
V C S   S i m u l a t i o n   R e p o r t
```

```
Time: 1723750 ps
```

### 4 설계한 Decoder & 설계한 Scheduler

```
[Result D] Duration: 140.000000 ps
```

```
[Result E] Duration: 435.000000 ps
```

```
=====
Test Completed.
```

```
=====
$finish called from file "/home/urp01/jungwoo/DDR2_Memory_Controller/script/./STEP2/SIM/TB/SAL_TB_TOP.sv", line 231.
```

```
$finish at simulation time 1721250.0 ps
```

```
V C S   S i m u l a t i o n   R e p o r t
```

```
Time: 1721250 ps
```